Information and Computation 297 (2024) 105133



Contents lists available at ScienceDirect

Information and Computation

journal homepage: www.elsevier.com/locate/yinco



Scooby: Improved multi-party homomorphic secret sharing based on FHE ☆



Ilaria Chillotti [a], Emmanuela Orsini [b,*,1], Peter Scholl [c,2], Barry Van Leeuwen [b,1]

[a] Zama, Paris, France
[b] imec-COSIC, KU Leuven, Leuven, Belgium
[c] Aarhus University, Aarhus, Denmark

ARTICLE INFO

Article history:
Received 23 January 2023
Received in revised form 17 August 2023
Accepted 22 December 2023
Available online 29 December 2023

Keywords:
HSS
FHE
Secure multiparty computation

ABSTRACT

In this paper we present new constructions of multi-party homomorphic secret sharing (HSS) based on a new primitive that we call *homomorphic encryption with decryption to shares* (HEDS). Our first scheme, which we call Scooby, is based on many popular fully homomorphic encryption (FHE) schemes with a linear decryption property. Scooby achieves an n-party HSS for general circuits with complexity $O(|F| + \log n)$, as opposed to $O(n^2 \cdot |F|)$ for the prior best construction based on multi-key FHE. Scooby relies on a trusted setup procedure, and can be based on (ring)-LWE with a super-polynomial modulus-to-noise ratio. In our second construction, Scrappy, assuming any generic FHE plus HSS for NC1-circuits, we obtain a HEDS scheme which does not require a super-polynomial modulus. While these schemes all require FHE, in another instantiation, Shaggy, we show how it is also possible to obtain multi-party HSS without FHE, instead relying on the DCR assumption to obtain 4-party HSS for constant-degree polynomials.

© 2023 Published by Elsevier Inc.

1. Introduction

One of the more interesting cryptographic constructions to be developed in recent years has been homomorphic secret sharing (HSS). This concept, which can be seen as a distributed analogue of homomorphic encryption, was introduced in [6], where a two party construction for branching programs was presented based on the decisional Diffie-Hellman assumption. The idea of HSS starts from the concept of a (traditional) secret sharing scheme, where an input x to some function is split into n shares, $(\mathbf{x}_1, \ldots, \mathbf{x}_n)$. This sharing, that in this work we always assume to be a full threshold sharing, is created via an

☆ This paper is an extended and revised version of a paper presented at the 13th Conference on Security and Cryptography for Networks. All the authors would like to thank Nigel P. Smart for his contribution on the conference version of this paper.

* Corresponding author.
E-mail addresses: chillotti.ilaria@gmail.com (I. Chillotti), emmanuela.orsini@kuleuven.be (E. Orsini), peter.scholl@cs.au.dk (P. Scholl), barry.vanleeuwen@kuleuven.be (B. Van Leeuwen).

[1] The work of authors from KU Leuven was supported by CyberSecurity Research Flanders with reference number VR20192203 and by the FWO under an Odysseus project GOH9718N. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of Cyber Security Research Flanders or the FWO.

[2] Peter Scholl was supported by the Independent Research Fund Denmark (DFF) under project number 0165-00107B (C3PO) and the Aarhus University Research Foundation (AUFF). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the DFF or AUFF.

https://doi.org/10.1016/j.ic.2023.105133
0890-5401/© 2023 Published by Elsevier Inc.

algorithm $(\mathbf{x}_1, \ldots, \mathbf{x}_n) \leftarrow \mathsf{Share}^{\mathsf{HSS}}(x)$. An HSS scheme has two additional algorithms, the first $y_i \leftarrow \mathsf{Eval}^{\mathsf{HSS}}(F; \mathbf{x}_i)$ takes a function description F and a share $\mathbf{x}_i$ and produces a corresponding output share y_i. The second $\mathsf{Rec}^{\mathsf{HSS}}(y_1, \ldots, y_n)$ takes the output shares and reconstructs the result $F(x)$. To avoid trivial solutions one requires that the length of the y_j's should be *compact*, i.e. it only depends on the output length of the function F and the security parameter. An important class of HSS schemes are those with *additive reconstruction*, where the function $\mathsf{Rec}^{\mathsf{HSS}}$ simply computes $y_1 + \ldots + y_n$. We refer to these as *additive HSS schemes*. It is such additive HSS schemes that we focus on in this work.

Motivation for HSS The main application of HSS is towards secure two-party or multi-party computation with succinct communication. Indeed, the breakthrough work of [6] showed that for a large class of circuits, it's possible to achieve secure computation with sublinear communication in the circuit size under DDH, which was previously only known using fully homomorphic encryption. Since then, HSS has proven useful in various other applications, and is closely related to pseudorandom correlation generators [3] and pseudorandom correlation functions [4], which allow generation of correlated randomness with a minimal amount of interaction. HSS for simple classes of functions, particularly the case of distributed point functions [21], has also proven useful for applications including private information retrieval [7] and secure RAM computation [17]. On a more theoretical side, HSS has also been used to build 2-round secure computation and nearly optimal worst-case to average-case reductions [8].

Additive reconstruction is an important feature of HSS in many secure computation settings, where it may be desirable for the output shares to be re-used in another secure computation based on secret sharing. This is the case, for instance, when using HSS to generate preprocessing material for multi-party computation protocols in the dishonest majority setting [3]. It can also be a useful feature in scenarios where a client reconstructing the output is constrained to perform only lightweight computations.

Current state of HSS and related primitives Related to HSS is the dual concept of function secret sharing (FSS) [5,7]. In FSS, the shared data is a secret function F (from some publicly known class of functions), such that the parties can locally obtain secret shares of $F(x)$, for any public input x. For general function classes such as polynomially-sized circuits, function secret sharing and homomorphic secret sharing are equivalent.

Obtaining efficient n-party HSS and FSS is complex for general functions. The most efficient known scheme is that based on an LWE-construction from spooky encryption. Spooky encryption, introduced by Dodis et al. [16], is a rather complex construction based on a multi-key variant of FHE [15,23], and for our purposes we are only interested in *additive-function-sharing* spooky encryption (or AFS-spooky encryption). Spooky encryption is a semantically secure public-key encryption scheme consisting of the usual three algorithms $(\mathsf{KeyGen}^{\mathsf{Spooky}}, \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}}, \mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}})$ as well as an additional algorithm, that is $\mathsf{Eval}^{\mathsf{Spooky}}_{\mathsf{pk}_1,\ldots,\mathsf{pk}_n}(F, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$. The $\mathsf{Eval}^{\mathsf{Spooky}}$ algorithm, given a function F on n arguments from a given class, and n ciphertexts ct_i, encrypting x_i under pk_i, produces n new ciphertexts $\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n$ such that, computing $y_i \leftarrow \mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}_i}(\mathsf{ct}_i)$, we have that $y_1 + \ldots + y_n = F(x_1, \ldots, x_n)$.

In [16], it is shown that it is possible to build FSS from AFS-spooky encryption. Roughly, to *share* an input function F the dealer first generates n AFS-spooky key pairs $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}^{\mathsf{Spooky}}(1^\lambda)$. The dealer also generates an n-out-of-n description of the function F, i.e. functions $F_i(x)$ such that $F(x) = F_1(x) + \ldots + F_n(x)$. Finally, the function secret sharing of the input function F is defined to be the tuple $F_i = (\mathsf{sk}_i, \mathsf{pk}_1, \ldots, \mathsf{pk}_n, \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}_1}(F_1), \ldots, \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}_n}(F_n))$.

To define the FSS evaluation we create a function C_x which takes as input the n additive shares of a function F, and evaluates it on the input x, which is hard-coded into C_x. By applying

$$\mathsf{Eval}^{\mathsf{Spooky}}_{\mathsf{pk}_1,\ldots,\mathsf{pk}_n}\left(C_x, \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}_1}(F_1), \ldots, \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}_n}(F_n)\right),$$

we obtain ciphertexts $\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n$, where ct'_i can be decrypted (using sk_i) to obtain y_i such that $y_1 + \ldots + y_n = F(x)$.

In [8], Boyle et al. showed how the FSS construction from spooky encryption can be modified to enable an additive HSS scheme. The $\mathsf{Share}^{\mathsf{HSS}}(x)$ operation additively shares x into $x = x_1 + \ldots + x_n$, generates n spooky key pairs $(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{KeyGen}^{\mathsf{Spooky}}(1^\lambda)$, and then encrypts x_i via $\mathsf{ct}'_i \leftarrow \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}_i}(x_i)$. The share values $\mathbf{x}_i$ output by $\mathsf{Share}^{\mathsf{HSS}}(x)$ being $\mathbf{x}_i = (\{\mathsf{pk}_i\}_{i=1}^n, \{\mathsf{ct}_i\}_{i=1}^n, \mathsf{sk}_i)$. The $\mathsf{Eval}^{\mathsf{HSS}}(F, \mathbf{x}_i)$ function executes $\mathsf{Eval}^{\mathsf{Spooky}}_{\mathsf{pk}_1,\ldots,\mathsf{pk}_n}$ on the function F and the ciphertext $(\mathsf{ct}_1, \ldots, \mathsf{ct}_n)$ so as to obtain n ciphertexts $\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n$. The output of $\mathsf{Eval}^{\mathsf{HSS}}(F, \mathbf{x}_i)$ then being $\mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}_i}(\mathsf{ct}'_i)$.

Thus, there is a strong connection between HSS, FSS and spooky constructions, and, as mentioned above, the prior most efficient n-party HSS and FSS constructions for circuits arise from AFS-spooky based on LWE (and a circular security assumption). The best current construction for AFS-spooky encryption of Dodis et al. [16] has a complexity of $O(n^2 \cdot |F|)$. In particular, each gate of the underlying arithmetic circuit F requires a bootstrapping operation which in the multi-key FHE setting has complexity $O(n^2)$.

1.1. Our contribution

We present new constructions of homomorphic secret sharing in the multi-party setting, supporting up to $n - 1$ out of n corruptions. Our constructions take a step forward from the only previous general construction, based on AFS-spooky

encryption [16], in two different ways: in some cases we propose a more efficient construction while in other cases we achieve the same functionality by relying on different assumptions.

HSS from Homomorphic Encryption with Decryption Shares. We present our constructions as a new primitive called *homomorphic encryption with decryption to shares (HEDS)*, which can be seen as a homomorphic encryption scheme with a special decryption algorithm that (non-interactively) outputs an n-party secret share of the encrypted message. HEDS is closely related to both spooky encryption and homomorphic secret sharing (HSS): the major difference compared to spooky is that HEDS needs to set up private decryption keys under a common public key with either a trusted setup algorithm or a secure multiparty computation protocol, while the difference with HSS is that the homomorphic evaluation algorithm is public. As is the case for spooky, HEDS immediately implies additive HSS for the same class of functions.

Scooby Construction: HEDS from Linear Decryption FHE. We show that HEDS can be built using any FHE scheme with a special decryption property, which we call *linear decryption based fully homomorphic encryption* (LD-based FHE) schemes. Examples of such LD-based FHE schemes are LWE-based constructions like BGV [11], BFV [18], GSW [20] and TFHE [13,14]. Notice this special property of almost all FHE schemes, where the decryption function is a linear function of the secret key, has been exploited previously, including for HSS in the two-party setting [16,9] and other applications [10,19].

Any of these schemes can be used to instantiate our Scooby construction, giving additive HSS for circuits. Recall in AFS-spooky the key generation is run independently by the n-parties, in our variation the keys are instead generated by a trusted third party.[3]

Since this construction only requires single-key FHE and not multi-key FHE, we obtain n-party HSS that is simpler and more efficient than the AFS-spooky-based construction. In particular, the computational complexity grows as $O(|F| + \log n)$, whereas AFS-spooky has complexity $O(n^2 \cdot |F|)$ for n parties. In addition, when instantiated with BGV we show that the standard parameter sets for bootstrapping are sufficient for our construction.

At a high level, at the core of Scooby is a well-known 2-party distributed decryption procedure, which non-interactively decrypts an LWE-based ciphertext into two shares, assuming the ciphertext modulus has super-polynomial size. This trick has been used previously, including in the construction of AFS-spooky. Our main contribution is to bootstrap this 2-party non-interactive algorithm into an n-party non-interactive algorithm. We do this by placing the n parties on the leaves of a binary tree, and then homomorphically evaluating the two party protocol at each internal node of the tree. Each party only needs to evaluate the 2-party protocol at each node on the path from the root to its leaf. Each homomorphic evaluation at the internal nodes is exactly equivalent to a bootstrapping operation, namely a homomorphic evaluation of the decryption circuit for some key. Thus, decryption into shares costs $O(\log n)$ operations per party.

Removing the Super-polynomial Modulus. The problem with Scooby, as well as all LWE-based additive HSS schemes, is that we require a super-polynomial modulus-to-noise ratio in the underlying LD-based FHE scheme. This is a stronger form of LWE assumption that usually requires larger parameters to compensate. We give a variant of the construction where we only need standard FHE, together with an HSS scheme for NC1 circuits. Using recent constructions of HSS [25,27,1] based on either Paillier encryption or class groups, we obtain the first additive HSS schemes for circuits that do not require LWE with a super-polynomial modulus. The complexity of the HSS is also $O(|F| + \log n)$. However, it is likely to be less efficient in practice than Scooby, and is also not secure against a quantum adversary. We call this construction Scrappy.

Avoiding FHE Entirely. We also show that in certain cases, we can obtain multi-party HSS without using any form of FHE whatsoever. We do this through a variant of the previous construction, where we bootstrap a HEDS scheme to handle more parties by homomorphically evaluating its own decryption circuit. This transformation is more challenging to apply without resorting to FHE, and we are only able to obtain a 4-party HEDS scheme for constant-degree polynomials, based on Paillier encryption. Nevertheless, as far as we are aware, this is the first instance of > 2-party, dishonest majority HSS for constant-degree polynomials, without relying on FHE. We call this construction Shaggy.

We summarize our results in Table 1.

Spooky from HEDS. In addition, in the full version, we show how our Scooby scheme can be adapted to give a true AFS-spooky encryption, i.e. with no trusted setup and independent keys, if we base our construction on *specific* multi-key FHE (MK-FHE). This instantiation can have a simpler complexity than that given in [16], in particular, assuming the function F can be evaluated without bootstrapping, our complexity is $O(n \cdot |F| + n^2 \cdot \log n)$. If F requires a bootstrapping for all the operations, it is $O(n^2 \cdot |F| + n^2 \cdot \log n)$. We call this construction Casper.

In the full version we give two variants of Casper, based on two different underlying FHE schemes. We note that being MK-FHE schemes, the construction will be less efficient than our Scooby scheme, which works over most (practical) FHE schemes. It is interesting to note that the spooky construction from [16] also goes via MK-FHE. In particular, they make use of the MK-FHE scheme of [15,23]. The route though is more complex than our tree-based construction, leading to an increased complexity.

[3] In some sense the "spooky" behaviour exhibited by spooky encryption cannot really be explained, whereas our "spooky" behaviour can be explained by the setup procedure. This setup procedure in some sense acts like the janitor in Scooby-Doo, who has set up the spooky goings-on.

Table 1
Summary of n-party HSS Constructions. All FHE-based constructions allow arbitrary functions F, and assume circular security to avoid blow-up in the key sizes (this assumption can be removed by relaxing to bounded-depth circuits). The asymptotic complexities of evaluation ignore potential factors in λ that are independent of n and F.

Construction	Assumptions	Setup	Complexity (per party)
DHRW [16] (AFS-Spooky)	LWE with super-polynomial modulus	Uniform CRS	$O(n^2 \cdot \|F\|)$
Scooby: §5 (HEDS)	LD-based FHE with super-polynomial modulus	Trusted	$O(\|F\| + \log n)$
Scrappy: §6.1 (HEDS)	Generic FHE + 2-party HSS for NC1	Trusted	$O(\|F\| + \log n)$
Shaggy: §6.2 (HEDS)	2-party HSS for NC1	Trusted	$O(\|F\|)$ ($n = 4$, constant-deg F)
Casper: (Full Version) (AFS-Spooky)	Specific MK-FHE with super-polynomial modulus	Uniform CRS	$O(n \cdot \|F\| + n^2 \cdot \log n)$ or $O(n^2 \cdot \|F\| + n^2 \cdot \log n)$

2. Preliminaries

For a set S, we denote by $a \leftarrow S$ the process of drawing a from S with a uniform distribution on the set S. If D is a probability distribution, we denote by $a \leftarrow D$ the process of drawing a with the given probability distribution. For a probabilistic algorithm A, we denote by $a \leftarrow A$ the process of assigning a the output of algorithm A; with the underlying probability distribution being determined by the random coins of A.

All reductions modulo an integer p will be assumed to be centred, i.e. in the interval $(-p/2, \ldots, p/2)$.

We let $R = \mathbb{Z}[X]/(X^N + 1)$ and R_p denote the localisation of R at p, i.e. $(\mathbb{Z}/p\mathbb{Z})[X]/(X^N + 1)$. For a real interval I we let R_I denote the restriction of the *set* R to have coefficients in the support of I. Thus as sets (but not as rings) we have $R_q = R_{(-q/2,\ldots,q/2)}$.

2.1. Homomorphic secret sharing

The following definition of public-key HSS is adapted from [9]. Note that, as we are only interested in schemes with additive reconstruction, we can disregard the decoding algorithm, $\mathsf{Dec}_{\mathsf{sk}}^{\mathsf{HSS}}$, that is given in the more general definition of HSS [8]. Concretely, in additive HSS the decoding algorithm simply adds up all the shares.

Definition 1 *(Additive public key homomorphic secret sharing).* An n-party, public-key *homomorphic secret sharing (HSS)* scheme for a class of functions $\mathcal{F}$ over a ring $\mathcal{R}$ with input space $\mathcal{I} \subseteq R$ consists of PPT algorithms $(\mathsf{KeyGen}^{\mathsf{HSS}}, \mathsf{Share}_{\mathsf{pk}}^{\mathsf{HSS}}, \mathsf{Eval}_{\mathsf{pk}}^{\mathsf{HSS}})$ with the following syntax:

- $\mathsf{KeyGen}^{\mathsf{HSS}}(1^\lambda, n) \to (\mathsf{pk}, (\mathsf{ek}_1, \ldots, \mathsf{ek}_n))$: Given a security parameter 1^λ, the setup algorithm outputs a public key pk and n evaluation keys $(\mathsf{ek}_1, \ldots, \mathsf{ek}_n)$.
- $\mathsf{Share}_{\mathsf{pk}}^{\mathsf{HSS}}(\mathsf{pk}, x) \to (\mathbf{x}_1, \ldots, \mathbf{x}_n)$: Given public key pk and private input value $x \in \mathcal{I}$, the share algorithm outputs shares $(\mathbf{x}_1, \ldots, \mathbf{x}_n)$.
- $\mathsf{Eval}_{\mathsf{pk}}^{\mathsf{HSS}}(F; \mathbf{x}_i, \mathsf{ek}_i) \to y_i$: On input a function $F \in \mathcal{F}$, the parties share $\mathbf{x}_i$, and it's evaluation key ek_i, the homomorphic evaluation algorithm outputs $y_i \in R$, which is party i's share of an output $y \in R$.

This definition is in the multi-input setting, meaning that it supports a compact evaluation of a function F on shares of inputs $x^{(1)}, \ldots, x^{(\rho)}$ given by ρ parties that are usually referred to as *clients*. More concretely, each client inputs $x^{(i)}$ to the Share algorithm which returns shares $\mathbf{x}_j^{(i)}$, $j \in [n]$, to n parties (the *servers*). Each server can then locally run Eval on input $(\mathbf{x}_j^{(1)}, \ldots, \mathbf{x}_j^{(\rho)})$ obtaining a share y_j such that $F(x^{(1)}, \ldots, x^{(\rho)}) = \sum_{j \in [n]} y_j$. Note that the $\mathsf{KeyGen}^{\mathsf{HSS}}$ algorithm cannot be run by any single party, so can be seen as a form of correlated randomness generated by a trusted dealer. We describe the required security properties for the algorithms $(\mathsf{KeyGen}, \mathsf{Share}, \mathsf{Eval})$ according to this more general formulation.

Definition 2 *(HSS (statistical) correctness).* We say that an n-party public-key HSS scheme $(\mathsf{KeyGen}^{\mathsf{HSS}}, \mathsf{Share}_{\mathsf{pk}}^{\mathsf{HSS}}, \mathsf{Eval}_{\mathsf{pk}}^{\mathsf{HSS}})$ is correct for a class of functions $\mathcal{F}$ if, for all security parameters $\lambda \in \mathbb{N}$, for all functions $F \in \mathcal{F}$, for all $x^{(1)}, \ldots, x^{(\rho)} \in \mathcal{I}$ (where $\mathcal{I}$ is the input space of F), for all $(\mathsf{pk}, \mathsf{ek}_1, \ldots, \mathsf{ek}_n) \leftarrow \mathsf{KeyGen}^{\mathsf{HSS}}(1^\lambda)$ and for all $(\mathbf{x}_1^{(i)}, \ldots, \mathbf{x}_n^{(i)}) \leftarrow \mathsf{Share}_{\mathsf{pk}}^{\mathsf{HSS}}(\mathsf{pk}, x^{(i)})$, $i \in [\rho]$, we have

$$\Pr\left[y_1 + \cdots + y_n = F(x^{(1)}, \ldots, x^{(\rho)}) \right] \geq 1 - \mathsf{negl}(\lambda),$$

Security Experiment $\mathsf{Exp}^{\mathsf{HSS,sec}}_{\mathcal{A},j}(\lambda)$

Let $I \subset [n]$ be the set of corrupt servers.

1. $(\mathsf{pk}, (\mathsf{ek}_1, \ldots, \mathsf{ek}_n)) \leftarrow \mathsf{KeyGen}^{\mathsf{HSS}}(1^\lambda)$.
2. $(x_0, x_1, \mathsf{state}) \leftarrow \mathcal{A}(1^\lambda)$.
3. $b \leftarrow \{0, 1\}$.
4. $(\mathbf{x}_{b,1}, \ldots, \mathbf{x}_{b,n}) \leftarrow \mathsf{Share}^{\mathsf{HSS}}_{\mathsf{pk}}(\mathsf{pk}, x_b)$.
5. $b' \leftarrow \mathcal{A}(\mathsf{state}, \mathsf{pk}, \{\mathsf{ek}_j, \mathbf{x}_{b,j}\}_{j \in I})$.
6. Return $b' = b$.

Fig. 1. Security Experiment $\mathsf{Exp}^{\mathsf{HSS,sec}}_{\mathcal{A},j}(\lambda)$.

with

$$y_j \leftarrow \mathsf{Eval}^{\mathsf{HSS}}_{\mathsf{pk}}(F; (\mathbf{x}_j^{(1)}, \ldots, \mathbf{x}_j^{(\rho)}), \mathsf{ek}_j), \ \ j \in [n],$$

where the probability is taken over the random coins of $\mathsf{KeyGen}^{\mathsf{HSS}}$, $\mathsf{Share}^{\mathsf{HSS}}_{\mathsf{pk}}$ and $\mathsf{Eval}^{\mathsf{HSS}}_{\mathsf{pk}}$.

Definition 3 *(HSS security).* Let I be the set of corrupt servers. For each $j \in I$ and non-uniform adversary $\mathcal{A}$ (of size polynomial in the security parameter λ), it holds that

$$\left| \Pr[\mathsf{Exp}^{\mathsf{HSS,sec}}_{\mathcal{A},j}(\lambda) = 1] \right| \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where $\mathsf{Exp}^{\mathsf{HSS,sec}}_{\mathcal{A},j}(\lambda)$ is the experiment defined in Fig. 1.

Remark 1 *(Private-key HSS).* HSS can also be defined in the single-input, private key setting, which is weaker than the public-key flavour described above. Here, there is no KeyGen algorithm, and Share is run only once on all inputs together, so can be seen as a trusted dealer algorithm that distributes the shares.

2.2. Spooky encryption

"Spooky" encryption is a type of public key encryption scheme which exhibits a form of limited malleability, so called "spooky action at a distance" [16], meaning that each party can locally compute one output of an n-output function, when given only encryptions of the inputs. The particular form of limited malleability we support is called *additive-function-sharing* spooky encryption (or AFS-spooky encryption), where the function computed will always be a random n-out-of-n secret sharing of a circuit C. We present a definition which works for any finite ring $\mathcal{R}$, and arithmetic circuit C, and not just for the case of $\mathbb{F}_2$ as originally presented.

Definition 4 *(AFS-spooky encryption).* An *AFS-spooky* encryption scheme, over a finite field $\mathbb{F}_p$, is a public-key encryption scheme given by a tuple of four algorithms $(\mathsf{KeyGen}^{\mathsf{Spooky}}, \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}}, \mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}}, \mathsf{Eval}^{\mathsf{Spooky}}_{\mathsf{pk}_1,\ldots,\mathsf{pk}_n})$ with the following syntax:

- $\mathsf{KeyGen}^{\mathsf{Spooky}}(1^\lambda) \mapsto (\mathsf{pk}, \mathsf{sk})$: This is a probabilistic polynomial time algorithm which on input a security parameter λ outputs a public/private key pair.
- $\mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}}(m) \mapsto \mathsf{ct}$: This probabilistic polynomial time algorithm takes a message $m \in \mathcal{R}$ and generates a ciphertext ct encrypting m under the public key pk.
- $\mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}}(\mathsf{ct}) \mapsto m$: Given a ciphertext ct encrypted under the public key associated to sk, this algorithm produces the underlying plaintext.
- $\mathsf{Eval}^{\mathsf{Spooky}}_{\mathsf{pk}_1,\ldots,\mathsf{pk}_n}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n) \mapsto (\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n)$: Given an arithmetic circuit description $C : \mathcal{R}^n \longrightarrow \mathcal{R}$, n public keys $\mathsf{pk}_1, \ldots, \mathsf{pk}_n$, and n of ciphertexts $\mathsf{ct}_1, \ldots, \mathsf{ct}_n$, this produces n ciphertexts $\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n$

An AFS-spooky encryption scheme must be correct, as an encryption scheme, i.e. we must have

$$\forall (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}^{\mathsf{Spooky}}(1^\lambda), \ \forall m \in \mathcal{R} \ : \ \mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}}(\ \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}}(m)\ ) = m.$$

It must also be IND-CPA as an encryption scheme and satisfy the following form of limited malleability called AFS-spooky correctness.

Definition 5 *(AFS-spooky correctness).* There exists a negligible function ν such that for all $\lambda \in \mathbb{N}$, every arithmetic circuit C computing a n-argument function $f : \mathcal{R}^n \longrightarrow \mathcal{R}$, and all inputs $x_1, \ldots, x_n$ of C, we have

$$\Pr\left[\sum_{i\in[n]} y_i = C(x_1,\ldots,x_n) : \begin{array}{l} \forall i \in [n], (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}^{\mathsf{Spooky}}(1^\lambda), \\ \forall i \in [n], \mathsf{ct}_i \leftarrow \mathsf{Enc}^{\mathsf{Spooky}}_{\mathsf{pk}}(x_i), \\ (\mathsf{ct}'_1,\ldots,\mathsf{ct}'_n) \leftarrow \mathsf{Eval}^{\mathsf{Spooky}}_{\mathsf{pk}_1,\ldots,\mathsf{pk}_n}(C,\mathsf{ct}_1,\ldots,\mathsf{ct}_n), \\ \forall i \in [n], y_i \leftarrow \mathsf{Dec}^{\mathsf{Spooky}}_{\mathsf{sk}_i}(\mathsf{ct}'_i) \end{array}\right] \geq 1-\nu(\lambda)$$

In [16], it is shown how to construct an AFS-spooky encryption scheme in the CRS model using an LWE-based multi-key FHE [15,23] and assuming a circular security assumption. The common reference string (output by a separate generation algorithm), necessary in the multi-key FHE construction, is assumed as input to the key generation algorithm, and correctness and security hold for all outputs of the common reference string generator.

In their work, Dodis et al. [16] show that AFS-spooky encryption implies FSS for general circuit. In [8], Boyle et al. show that AFS-spooky also enables HSS for multiple inputs; in fact, it implies HSS without any setup, where the key generation algorithm is simply run locally by each client providing input.

3. Homomorphic encryption with decryption to shares (HEDS)

In this section we formally introduce the notion of a scheme which implements Homomorphic Encryption with Decryption to Shares (HEDS) and relate it with other concepts described in previous sections. Loosely speaking, a HEDS encryption scheme is similar to public-key HSS, except with a public evaluation algorithm that outputs a ciphertext, more akin to evaluation in homomorphic encryption. The ciphertext is then converted into shares in the decryption algorithm, which uses one party's private key. In addition, similarly to HSS, but unlike in spooky encryption, the parties need to engage in a protocol, or assume a trusted third party, to set up the associated public and secret keys. Thus the action from the outside seems spooky, but this can be explained away as an effect of the setup protocol.

We start by giving the definition of HEDS, and then we show that it enables both homomorphic and function secret sharing.

Definition 6 *(HEDS encryption).* A *HEDS encryption scheme* for a class of functions $\mathcal{F} : \mathcal{R}^* \to \mathcal{R}$, over a ring $\mathcal{R}$, is given by a tuple of PPT algorithms $(\mathsf{SetUp}^{\mathsf{HEDS}}, \mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}, \mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{sk}}, \mathsf{Eval}^{\mathsf{HEDS}}_{\mathsf{pk}})$, with the following syntax:

- $\mathsf{SetUp}^{\mathsf{HEDS}}(1^\lambda, n) \mapsto (\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$: This randomized algorithm takes as input a security parameter λ, a number of parties n. It outputs a public key, pk, and a set of secret key shares, $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$.
- $\mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}(m) \mapsto \mathsf{ct}$: This takes as input the public key and a message $m \in \mathcal{R}$, and outputs a ciphertext.
- $\mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{sk}_i}(\mathsf{ct}) \mapsto y_i$: Given a ciphertext ct encrypted under the public key, and a secret key share sk_i, this outputs a message share y_i.
- $\mathsf{Eval}^{\mathsf{HEDS}}_{\mathsf{pk}}(C, (\mathsf{ct}_1, \ldots, \mathsf{ct}_\rho)) \mapsto \mathsf{ct}$: On input of the public key pk, a set of n ciphertexts, and an arithmetic circuit description $C : \mathcal{R}^\rho \longrightarrow \mathcal{R}$ of a function from the specified class, this produces a ciphertext ct.

The algorithms $\left(\mathsf{SetUp}^{\mathsf{HEDS}}, \mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}, \mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{sk}}, \mathsf{Eval}^{\mathsf{HEDS}}_{\mathsf{pk}}\right)$ should satisfy the following correctness and security requirements.

Definition 7 *(HEDS correctness).* There exists a negligible function ν such that for all $\lambda \in \mathbb{N}$, every arithmetic circuit C computing a ρ-argument function $f : \mathcal{R}^\rho \longrightarrow \mathcal{R}$ in $\mathcal{F}$, and all inputs $x_1, \ldots, x_\rho$ of C, we have

$$\Pr\left[\sum_{i\in[n]} y_i = C(x_1,\ldots,x_\rho) \ : \begin{array}{l} (\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{SetUp}^{\mathsf{HEDS}}(1^\lambda, n), \\ \forall i \in [\rho], \mathsf{ct}_i \leftarrow \mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}(x_i), \\ \mathsf{ct} \leftarrow \mathsf{Eval}^{\mathsf{HEDS}}_{\mathsf{pk}}(C, (\mathsf{ct}_1, \ldots, \mathsf{ct}_\rho)), \\ \forall i \in [n], y_i \leftarrow \mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{sk}_i}(\mathsf{ct}) \end{array}\right] \geq 1-\nu(\lambda).$$

Definition 8 *(HEDS security).* For all subsets $A \subset [n]$ of size $< n$, and all probabilistic polynomial time adversaries $(\mathcal{A}_1, \mathcal{A}_2)$ we have

$$\Pr\left[b = b' \ : \begin{array}{l} (\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{SetUp}^{\mathsf{HEDS}}(1^\lambda, n), b \in \{0,1\}, \\ (m_0, m_1, \mathsf{state}) \leftarrow \mathcal{A}_1(\mathsf{pk}, \{\mathsf{sk}_i\}_{i\in A}), \\ \mathsf{ct} \leftarrow \mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}(m_b), \\ b' \leftarrow \mathcal{A}_2(\mathsf{ct}, \mathsf{state}) \end{array}\right] \leq \mathsf{negl}(\lambda),$$

i.e. the encryption scheme is IND-CPA, even when up to $n-1$ secret keys are given to the adversary.

Compactness Just as with fully homomorphic encryption, we say that HEDS is *compact* if the share decryption algorithm is independent of the evaluated function.

3.1. Multi-input HSS from HEDS encryption

Here we relate HEDS encryption and HSS showing that HEDS encryption implies HSS with multiple inputs. Let $\mathcal{P}$ be a set of n servers and $\mathcal{C}$ be a set of m clients. Let C be a circuit representing a function $F : \mathcal{R}^m \to \mathcal{R}$ in a class function $\mathcal{F}$. To build an HSS-scheme, we need to define three algorithms $\mathsf{KeyGen}^{\mathsf{HSS}}$, $\mathsf{Share}^{\mathsf{HSS}}$, $\mathsf{Eval}^{\mathsf{HSS}}$ as in Definition 1. Let $(\mathsf{SetUp}^{\mathsf{HEDS}}, \mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}, \mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{sk}}, \mathsf{Eval}^{\mathsf{HEDS}}_{\mathsf{pk}})$ be a HEDS encryption scheme for $\mathcal{F}$, as defined in the previous section, we proceed as follows.

- $\mathsf{KeyGen}^{\mathsf{HSS}}(1^\lambda, n)$:
  1. Run $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{SetUp}^{\mathsf{HEDS}}(1^\lambda, n)$
  2. For each $i \in [n]$, set $\mathsf{ek}_i := \mathsf{sk}_i$
  3. Return pk and $(\mathsf{ek}_1, \ldots, \mathsf{ek}_n)$
- $\mathsf{Share}^{\mathsf{HSS}}_{\mathsf{pk}}(x^{(j)})$: Each client $P_j \in \mathcal{P}$, on input $x^{(j)}$ performs the following steps. We recall that the goal is to obtain shares $(\mathbf{x}_1, \ldots, \mathbf{x}_n)$ of $(x^1, \ldots, x^m)$.
  1. For $i \in [n]$ and $j \in [m]$, generate $x_i^{(j)}$ such that $x^{(j)} = x_1^{(j)} + \ldots + x_n^{(j)}$.
  2. For each $x_i^{(j)}$, compute $\mathsf{ct}_i^{(j)} = \mathsf{Enc}^{\mathsf{HEDS}}_{\mathsf{pk}}(x_i^{(j)})$.
  3. Set $\mathbf{x}_i = \{\mathsf{ct}_i^{(j)}\}_{j \in [m]}$, for $i \in [n]$.
- $\mathsf{Eval}^{\mathsf{HSS}}_{\mathsf{pk}}(F; \mathbf{x}_i, \mathsf{ek}_i)$: Given a function $F : \mathcal{R}^m \to \mathcal{R}$, each server $i \in [n]$ fixes a circuit description C of F and proceeds as follows.
  1. Parse $\mathbf{x}_i$ as $\{\mathsf{ct}_i^{(j)}\}_{j \in [m]}$
  2. Compute $\mathsf{ct}_i = \mathsf{Eval}^{\mathsf{HEDS}}_{\mathsf{pk}}\left(C, (\mathsf{ct}_i^{(1)}, \ldots, \mathsf{ct}_i^{(m)})\right)$
  3. Compute $y_i = \mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{ek}_i}(\mathsf{ct}_i)$

By Definition 7, we know that the evaluation algorithm outputs to the servers the shares $y_1, \ldots, y_n$ such that $\sum_{i \in [n]} y_i = y = F(x^{(1)}, \ldots, x^{(m)})$.

Proposition 1. *Assuming the existence of a HEDS encryption scheme for a class of functions $\mathcal{F}$, there exists a public-key multi-input HSS scheme for $\mathcal{F}$.*

Proof. Correctness follows by inspection of the scheme described above and by correctness of the underlying HEDS construction. Security also follows from the security of HEDS. □

In the other direction, we observe that a public-key HSS scheme implies HEDS for the same class of functions, however, the resulting HEDS scheme may not be compact. This is because the HSS evaluation algorithm will have to be carried out in the HEDS decryption step, since HSS uses a private key for evaluation.

4. Linear-decryption based FHE

Our main constructions are based on a form of FHE which comes from LWE-style systems. We abstract much of the details of the specific construction away in what follows, for example the specific key generation and encryption algorithms. This allows us to capture schemes as diverse as BGV [11], BFV [18], GSW [20] and TFHE [13,14]. These schemes all have the same form of decryption equation, namely one based on a linear inner product combination of the ciphertext with the secret key, modulo the ciphertext modulus. The result of this inner product is then processed to produce the plaintext (which is an element of R_p for some prime p) in one of two distinct ways, depending on whether the message is embedded at the top of the range modulo q (as in FV), or the bottom of the range modulo q (as in BGV). We refer to these two types of decryption as FHE as being of type msb and type lsb respectively. We call the whole class of such FHE systems Linear Decryption based, or LD-based FHE. Similar definitions have been considered previously [9,19,10].

Let sec denote some statistical security parameter and λ denote a computational security parameter. We define such a scheme as follows, the precise encryption and evaluation algorithms are not important for our discussion.

Definition 9 *(LD-based FHE).* An *LD-based FHE* scheme is given by a tuple of algorithms $(\mathsf{KeyGen}^{\mathsf{FHE}}, \mathsf{Enc}^{\mathsf{FHE}}_{\mathsf{pk}}, \mathsf{Dec}^{\mathsf{FHE}}_{\mathsf{sk}}, \mathsf{Eval}^{\mathsf{FHE}}_{\mathsf{pk}})$, as follows:

- $\mathsf{KeyGen}^{\mathsf{FHE}}(1^\lambda, p) \mapsto (\mathsf{pars}, \mathsf{pk}, \mathsf{sk})$: This randomized algorithm takes as input the security parameter λ and a plaintext modulus space p. It outputs a tuple $(\mathsf{pars}, \mathsf{pk}, \mathsf{sk})$, where $\mathsf{pars} = (q, N, B, d, \Delta, S)$. The value q will correspond to the

ciphertext modulus,[4] the value N will be the LWE-ring dimension (which for convenience we assume is a power of two), the value B will be a "noise bound", the value d is one less than the dimension of the ciphertext space, the value Δ is set to be $\lfloor q/p \rfloor$, the value S is a bound on the secret key size S, and pk (resp. sk) will be the public (resp. private) keys.

The private key $\mathsf{sk} = (s_1, \ldots, s_d)$ is assumed to be a random element in R_q^d sampled such that $\|\mathsf{sk}\|_\infty \leq S$. Note, this is not necessarily sampled uniformly at random subject to this constraint.

All subsequent algorithms are assumed to take pars implicitly as input parameters.

- $\mathsf{Enc}_{\mathsf{pk}}^{\mathsf{FHE}}(m, \mathsf{type}) \mapsto \mathsf{ct}$: On input of $m \in R_p$ this will output a ciphertext $\mathsf{ct} \in R_q^{d+1}$ such that

$$\mathsf{ct} \cdot (1, -\mathsf{sk}) = \begin{cases} m + p \cdot \epsilon \pmod q & \text{If } \mathsf{type} = \mathsf{lsb}, \\ \Delta \cdot m + \epsilon \pmod q & \text{If } \mathsf{type} = \mathsf{msb}. \end{cases}$$

  A ciphertext such that $\|\epsilon\|_\infty \leq B$ will be called *valid*. The encryption algorithm produces such a valid ciphertext. The precise algorithm use for encryption will depend on the public key, and the specific scheme. All that concerns us is the form of the ciphertext.
- $\mathsf{Eval}_{\mathsf{pk}}^{\mathsf{FHE}}(F(x_1, \ldots, x_\ell), \{\mathsf{ct}_1, \ldots, \mathsf{ct}_\ell\}) \mapsto \mathsf{ct}$: On input of ℓ valid ciphertexts ct_i (of the same type) and an arithmetic function $F(x_1, \ldots, x_\ell)$ this function will homomorphically evaluate the function F over the ciphertexts, producing a valid ciphertext as output.
- $\mathsf{Dec}_{\mathsf{sk}}^{\mathsf{FHE}}(\mathsf{ct}) \mapsto m$: On input of a valid ciphertext and a secret key this will compute the message as

$$m = \begin{cases} (\mathsf{ct} \cdot (1, -\mathsf{sk}) \pmod q) \pmod p & \text{If } \mathsf{type} = \mathsf{lsb}, \\ \left\lfloor (\mathsf{ct} \cdot (1, -\mathsf{sk}) \pmod q) \cdot p/q \right\rceil & \text{If } \mathsf{type} = \mathsf{msb}. \end{cases}$$

The correctness requirement simply says that $\mathsf{Eval}^{\mathsf{FHE}}$, when given ℓ valid ciphertexts, outputs a valid encryption of the correct result. The security requirement is the standard notion of IND-CPA security.

For example: In the case of the BGV scheme [11] from ring-LWE we will have that $\mathsf{ct} = (c_0, c_1)$, so that decryption is given by $\mathsf{ct} \cdot (1, -\mathsf{sk}) = c_0 - s_1 \cdot c_1$, and, hence, for this scheme we have $n = 1$ and $\mathsf{sk} = s_1$. The BFV scheme [18] has the same structure, the main difference being that BFV uses the msb decryption, while BGV uses lsb.

In the case of Ring-GSW, a ciphertext is in $R_q^{(d+1)\times(d+1)\ell}$, with $d = 1$ and $\mathsf{sk} = s_1$. In practice it is composed by $2 \cdot \ell$ FV-like ciphertext (i.e., with the message encrypted in the msb). To decrypt a Ring-GSW ciphertext, we only decrypt one of these ciphertexts: the others contain redundant information. Another way of seeing a Ring-GSW ciphertext, is with a very sparse secret key $\mathsf{sk} = (\mathsf{sk}_1, \ldots, \mathsf{sk}_{2\ell})$, where all the keys corresponding to the FV-like ciphertext that we are not going to decrypt are set to zero. The TFHE scheme [13,14] uses a combination of FV-like ciphertexts (with message encrypted in the msb, called LWE and RLWE ciphertexts) and Ring-GSW ones.

Parameters for decryption to shares For such LD-based FHE schemes we have a special form of non-interactive two party distributed decryption, which we shall now outline in both the lsb and the msb case. We will require the parameters are selected so that

$$q > 2 \cdot p \cdot (B + 1) \cdot 2^{\mathsf{sec}}, \tag{1}$$

where sec is the statistical security parameter. This two-party distributed decryption, which is essentially the same technique as in [16,9], will form the basis of our first multi-party HEDS construction in Section 5.

4.1. Two-party distributed decryption: type lsb

Suppose sk is split into two keys sk_1 and sk_2 with $\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 \pmod q$, with sk_1 held by party P_1 and sk_2 held by party P_2. Now we can, without interaction, given a valid ciphertext ct encrypting a message m, compute an additive sharing of $m = m_1 + m_2 \pmod p$ between P_1 and P_2 as follows. We require that the parties have agreed upon a public random value for each decryption, but later will remove this using a PRF.

2-party DistDec^{lsb}: Let $R \leftarrow \mathbb{Z}_q$ be a public random nonce.

1. P_1 computes $d_1 \leftarrow \mathsf{ct} \cdot (1, -\mathsf{sk}_1) + R \pmod q$ and then $m_1 \leftarrow d_1 \pmod p$.
2. P_2 computes $d_2 \leftarrow \mathsf{ct} \cdot (0, -\mathsf{sk}_2) - R \pmod q$ and then $m_2 \leftarrow d_2 \pmod p$.

We prove that this leads to a correct result with overwhelming probability.

[4] In practice there may be many ciphertext moduli depending on which level a ciphertext is sitting at, at a high level this can be ignored. Although it can be important in practice.

Proposition 2. *Given an LD-based FHE scheme of type* lsb *(Definition 9), where* $(q, N, B, d, \Delta, S, \mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}^{\mathsf{FHE}}(1^\lambda, p)$*, with* $q > 2 \cdot p \cdot (B+1) \cdot 2^{\mathsf{sec}}$ *and* $\mathsf{sk}_1 + \mathsf{sk}_2 = \mathsf{sk}$*. Let* (ct, m) *be a pair of ciphertext/plaintext messages and* m_1 *and* m_2 *values obtained with the 2-party distributed decryption procedure described above. Then, it holds that*

$$m = m_1 + m_2 \pmod{p},$$

with probability at least $1 - N \cdot 2^{-\mathsf{sec}}$.

Proof. First we notice that

$$m = ((d_1 + d_2) \pmod{q}) \pmod{p},$$

and that we will always have $m = m_1 + m_2 \pmod{p}$ if the internal reduction modulo q in the decryption equation for m does not need to compensate for a wrap around. However, since we know ct is valid (i.e., $\mathsf{ct} \cdot (1, -\mathsf{sk}) = m + p \cdot \epsilon \pmod{q}$ with $\|\epsilon\|_\infty \leq B$) we also know that the coefficients of $d_1 + d_2 \pmod{q}$ will lie in the range $(-p \cdot (B+1), \ldots, p \cdot (B+1))$. Thus, the distributed decryption will potentially result in an error if and only if the coefficients of d_1 lie in one of the two ranges $(-q/2, -q/2 + p \cdot (B+1))$ or $(q/2 - p \cdot (B+1), q/2)$. Since each party added or subtracted the random R, it holds that d_1 is uniformly distributed in the range $(-q/2, \ldots, q/2)$. Therefore, the probability there is a wraparound in a single coefficient is bounded by $2 \cdot p \cdot (B+1)/q < 2^{-\mathsf{sec}}$. However, we also known that, if there is a wrap around, it will definitely result in an invalid distributed decryption, as the error only consists of the addition of a single value of $q \pmod{p} \neq 0$. Thus, a single coefficient will be correct with probability $1 - 2^{-\mathsf{sec}}$. To obtain a correct decryption we need all coefficients to be correct, which will happen with probability

$$\left(1 - 2^{-\mathsf{sec}}\right)^N \approx 1 - N \cdot 2^{-\mathsf{sec}}. \quad \square$$

4.2. Two party distributed decryption: type msb

To prove our result for the msb-type encryption schemes, we will use the following lemma.

Lemma 1 *([9], Lemma 1). Let* $p, q \in \mathbb{N}$*,* $R = \mathbb{Z}[X]/(X^N + 1)$ *for* N *a power of* 2*. Let* $m, e \in R_p$ *with* $||e||_\infty \leq B$*, and* $t_0, t_1 \in R_q$ *be uniformly random subject to* $t_0 + t_1 = (q/p) \cdot m + e \mod q$ *Then, it holds that*

$$\left\lfloor (p/q) \cdot t_0 \right\rceil + \left\lfloor (p/q) \cdot t_1 \right\rceil = m \mod p,$$

with probability at least $1 - N \cdot (B+1) \cdot p/q$ *over the choice of the shares* t_0, t_1*.*

As done before, we suppose that sk is split into two keys sk_1 and sk_2 such that P_1 holds sk_1 and P_2 holds sk_2, and that $\mathsf{sk} = \mathsf{sk}_1 + \mathsf{sk}_2 \pmod{q}$. We can then, without interaction, compute an additive sharing of $m = m_1 + m_2$ given a valid ciphertext ct encrypting m. This can be achieved as follows.

2-party DistDec^{msb}: Let $R \leftarrow \mathbb{Z}_q$ be a public random nonce.

1. P_1 computes $d_1 \leftarrow \mathsf{ct} \cdot (1, -\mathsf{sk}_1) \pmod{q}$ and then $m_1 \leftarrow \lfloor d_1 \cdot p/q \rceil$.
2. P_2 computes $d_2 \leftarrow \mathsf{ct} \cdot (0, -\mathsf{sk}_2) \pmod{q}$ and then $m_2 \leftarrow \lfloor d_2 \cdot p/q \rceil$.

We claim that this correctly computes $m = m_1 + m_2$ with overwhelming probability.

Proposition 3. *Given an LD-based FHE scheme of type* msb *(Definition 9), where* $(q, N, B, d, \Delta, S, \mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}^{\mathsf{FHE}}(1^\lambda, p)$*, with* $q > 2 \cdot p \cdot (B+1) \cdot 2^{\mathsf{sec}}$ *and* $\mathsf{sk}_1 + \mathsf{sk}_2 = \mathsf{sk}$*. Let* (ct, m) *be a pair of ciphertext/plaintext messages and* m_1 *and* m_2 *values obtained with the procedure described above. Then, it holds that*

$$m = m_1 + m_2 \pmod{p},$$

with probability at least $1 - N \cdot 2^{\mathsf{sec}}$.

Proof. By the assumption on B, namely Equation (1), and by Lemma 1, we have

$$1 - \frac{N \cdot (B+1) \cdot p}{q} \leq 1 - \frac{N \cdot (B+1) \cdot p}{2 \cdot p \cdot (B+1) \cdot 2^{\mathsf{sec}}} \leq 1 - N \cdot 2^{-\mathsf{sec}}.$$

This concludes the proof. $\square$

5. Scooby: multi-party HEDS from LD-based FHE

Here we detail how to construct a HEDS encryption scheme for the underlying ring R_p, from generic LD-based FHE. We call our construction Scooby, as it is similar to a spooky encryption but with a trusted setup. To denote the specific nature of this construction we refer to $\mathsf{SetUp}^{\mathsf{Scooby}}$, $\mathsf{Enc}_{\mathsf{pk}}^{\mathsf{Scooby}}$, etc., instead of $\mathsf{SetUp}^{\mathsf{HEDS}}$, $\mathsf{Enc}_{\mathsf{pk}}^{\mathsf{HEDS}}$, etc.

At the core of Scooby is the 2-party distributed decryption procedure described in the previous section. We show that, assuming an LD-based FHE scheme, this directly yields a 2-party Scooby. We then show how to bootstrap the 2-party scheme to the multi-party setting.

5.1. HEDS key generation

First, we need to slightly modify the KeyGen algorithm for the underlying FHE scheme to take a "special" form that is common to all standard FHE constructions. More concretely, the algorithm $\mathsf{KeyGen}^{\mathsf{FHE}}(1^\lambda, p)$ proceeds as follows, using two sub-procedures ParamGen() and PubKeyGen():

1. $\mathsf{params} \leftarrow \mathsf{ParamGen}(1^\lambda, p)$: This algorithm takes as input a security parameter λ, a plaintext modulo p and produces the scheme parameters $\mathsf{params} = (q, N, B, d, \Delta, S)$.
2. $\mathsf{sk} \leftarrow R_q^n$ such that $\|\mathsf{sk}\|_\infty \leq S$.
3. $\mathsf{pk} \leftarrow \mathsf{PubKeyGen}(1^\lambda, \mathsf{sk}, \mathsf{params})$: This algorithm, on input the secret key and scheme parameters, samples and outputs an associated public key pk.

5.2. Security assumption

In our construction, we generate an FHE public key based on a secret-key $\mathsf{sk} = \mathsf{sk}_0 + \mathsf{sk}_1$, where $\mathsf{sk}_0, \mathsf{sk}_1$ are both sampled uniformly with coefficients bounded by the parameter S. For security, we require that the scheme defined by $(\mathsf{pk}, \mathsf{sk})$ satisfies the standard IND-CPA security notion, even when the adversary is given one of the original secret keys sk_i. This is formalized as follows.

Definition 10 *(Bounded secret key IND-CPA security).* Let $\mathsf{FHE} = (\mathsf{KeyGen}^{\mathsf{FHE}}, \mathsf{Enc}_{\mathsf{pk}}^{\mathsf{FHE}}, \mathsf{Dec}_{\mathsf{sk}}^{\mathsf{FHE}}, \mathsf{Eval}_{\mathsf{pk}}^{\mathsf{FHE}})$ be a linear decryption-based FHE scheme, where $\mathsf{KeyGen}^{\mathsf{FHE}}$ is split into two sub-routines ParamGen, PubKeyGen as above.

We require that for $(q, N, B, d, \Delta, S) \leftarrow \mathsf{ParamGen}(1^\lambda, p)$, and $\mathsf{sk}_0, \mathsf{sk}_1 \leftarrow R_q^d$ with $\|\mathsf{sk}_i\| \leq S$, $\mathsf{sk} = \mathsf{sk}_0 + \mathsf{sk}_1$ and $\mathsf{pk} \leftarrow \mathsf{PubKeyGen}(\mathsf{sk})$, it holds that for any PPT algorithm $\mathcal{A}$, for any $\sigma \in \{0,1\}$, messages m_0, m_1 and bit $b \leftarrow \{0,1\}$:

$$\Pr[\mathcal{A}(1^\lambda, \mathsf{pk}, \mathsf{sk}_\sigma, \mathsf{Enc}_{\mathsf{pk}}^{\mathsf{FHE}}(m_b)) = b] \leq 1/2 + \mathsf{negl}(\lambda).$$

It is straightforward to verify that, given a linear decryption-based FHE scheme that satisfies the bounded secret-key IND-CPA security, we obtain a 2-party Scooby encryption scheme using the prior algorithms for 2-party distributed decryption into shares described in the previous section. Indeed, this 2-party distributed decryption forms the basis of the 2-party spooky construction in [16] and HSS construction in [9]. However, to obtain an n-party generalization is not immediate. A direct application of the trick used for 2-party to, say, 3-parties results in decryption errors due to unaccounted for wrap-arounds in the reduction modulo q of the local decryption. Coping with these wrap-arounds, without resorting to interaction, thus seems a challenge. A challenge which we solve in the next section.

5.3. From 2-party to n-party HEDS

Here we give the details of our construction Scooby, for n-party HEDS. The encryption and evaluation algorithms of Scooby are identical to that of the underlying linear decryption FHE scheme, so here we only describe the setup and share decryption procedures. We give two different variants of the construction, depending on whether the FHE scheme encodes the message in the lsb or msb of the ciphertext. In this section, we focus on a linear decryption FHE scheme that encodes the message in the lsb of the ciphertext; in the full version, we give a variant for the msb type.

Scooby *Setup.* Recall that the setup algorithm in HEDS takes as input a security parameter and outputs a global public key pk, as well as secret keys $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ to each of the n parties. In Fig. 2 we present a toy example with $n = 4$ to simplify the understanding. For Scooby, in both the lsb and msb variants of LD-based FHE scheme, the underlying $\mathsf{SetUp}^{\mathsf{Scooby}}$ algorithm is the same. Note that in the following, the $\mathsf{SetUp}^{\mathsf{Scooby}}$ algorithm should be seen as a trusted setup procedure that is either run by a trusted third party, or executed via an MPC protocol, which can be done, for instance, based on the techniques from [26].

The $\mathsf{SetUp}^{\mathsf{Scooby}}$ algorithm is described in Fig. 3. Recall that the main challenge is to setup up some key material which allows n parties to convert an FHE ciphertext into shares of the message, while using the 2-party distributed decryption method from the previous section. We build a binary tree with n leaves, where the original FHE ciphertext lives at the

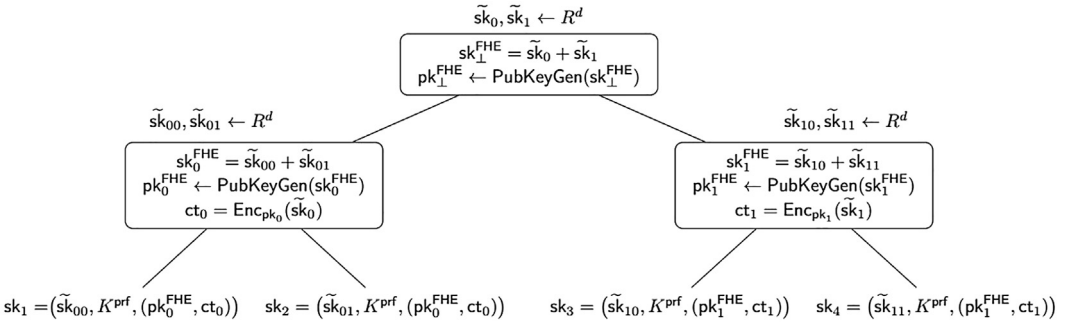


Fig. 2. Scooby setup for $n = 4$.

Algorithm $\mathsf{SetUp}^{\mathsf{Scooby}}(\lambda, p, n)$

The algorithm takes as input the security parameter λ, plaintext modulus p, and number of parties n. It outputs a public key pk and secret keys $(\mathsf{sk}_1, \ldots, \mathsf{sk}_n)$.

1. Let $\mathsf{params} = (q, N, B, d, \Delta, S) \leftarrow \mathsf{ParamGen}(1^\lambda, p)$.
2. Sample a key $K^{\mathsf{prf}} \leftarrow \{0,1\}^\lambda$.
3. We construct a complete (but not necessarily full at the last layer) binary tree with n leaves and height $h = \lceil \log(n) \rceil$, and index the levels from 0 up to h. Each node in level i of the tree is labelled with a string of i bits, so the root is the empty string $\perp$, and the children of node v are $v\|0$ and $v\|1$.
4. Sample $\widetilde{\mathsf{sk}}_0, \widetilde{\mathsf{sk}}_1 \leftarrow R_q^d$ such that $\|\widetilde{\mathsf{sk}}_j\|_\infty \leq S$.
5. Let $\mathsf{sk}_\perp^{\mathsf{FHE}} = \widetilde{\mathsf{sk}}_0 + \widetilde{\mathsf{sk}}_1$ and sample $\mathsf{pk}_\perp^{\mathsf{FHE}} = \mathsf{PubKeyGen}(1^\lambda, \mathsf{sk}_\perp^{\mathsf{FHE}}, \mathsf{params})$.
6. For each internal node v (excluding the root and leaves) with children $v\|0$ and $v\|1$:
   (a) Sample $\widetilde{\mathsf{sk}}_{v\|0}, \widetilde{\mathsf{sk}}_{v\|1} \leftarrow R_q^d$ such that $\|\widetilde{\mathsf{sk}}_j\|_\infty \leq S$.
   (b) Let $\mathsf{sk}_v^{\mathsf{FHE}} = \widetilde{\mathsf{sk}}_{v\|0} + \widetilde{\mathsf{sk}}_{v\|1}$, sample $\mathsf{pk}_v^{\mathsf{FHE}} = \mathsf{PubKeyGen}(1^\lambda, \mathsf{sk}_v^{\mathsf{FHE}}, \mathsf{params})$.
   (c) Let $\mathsf{ct}_v = \mathsf{Enc}_{\mathsf{pk}_v}^{\mathsf{FHE}}(\widetilde{\mathsf{sk}}_v)$.
7. Let sk_i contain the leaf secret key $\widetilde{\mathsf{sk}}_i$, together with K^{prf} and the public keys and ciphertexts on the path from the root to leaf i.
8. Output $\mathsf{pk} := \mathsf{pk}_\perp^{\mathsf{FHE}}$ and the secret keys $(\mathsf{sk}_1, \ldots, \mathsf{sk}_n)$.

Fig. 3. Trusted setup algorithm for the Scooby construction.

root node. We split the FHE secret key $\mathsf{sk}^{\mathsf{FHE}}$ into two shares $\widetilde{\mathsf{sk}}_0$, $\widetilde{\mathsf{sk}}_1$, and then generate a fresh FHE key pair for each of the two child nodes, and encrypt each $\widetilde{\mathsf{sk}}_b$, for $b \in \{0,1\}$, under the corresponding public key. This process is repeated with the FHE secret keys generated for the children, and so on throughout the tree. Note that we abuse notation by writing $\mathsf{ct}_v = \mathsf{Enc}_{\mathsf{pk}_v}^{\mathsf{FHE}}(\widetilde{\mathsf{sk}}_v)$, even though $\widetilde{\mathsf{sk}}_v$ may not lie in the plaintext space; we implicitly assume here that $\widetilde{\mathsf{sk}}_v$ is broken up into bits (or possibly larger chunks), so ct_v is actually a vector of ciphertexts encrypting each bit separately.

The idea is that, during the decryption phase, the parties can homomorphically evaluate the 2-party distributed decryption function at each node of the tree, obtaining a share of the message, now encrypted under a child node's public key. The i-th party repeats this for each node on the path to leaf i, where it finally obtains a ciphertext encrypting an n-party sharing of the original message, which it can decrypt.

Given this setup procedure we define $\mathsf{Enc}_{\mathsf{pk}}^{\mathsf{Scooby}}$ and $\mathsf{Eval}_{\mathsf{pk}}^{\mathsf{Scooby}}$ exactly as is the case in the underlying LD-based FHE scheme. Next, we detail the $\mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scooby}}$ procedure in the lsb case.

Scooby *Decryption.* The decryption algorithms for Scooby in the lsb/msb-mode are described in Fig. 4 and the full version, respectively. The decryption algorithm requires $\lceil \log n \rceil - 1$ evaluations of the $\mathsf{Eval}^{\mathsf{FHE}}$ function for the underlying LD-based FHE scheme, each for a different public key. Note that the circuit used in $\mathsf{Eval}^{\mathsf{FHE}}$ is almost exactly the decryption circuit, so the complexity of each of these homomorphic operations is the same as a bootstrapping operation in the underlying FHE scheme.

It is also clear that, due to the fact that at each internal branch we are homomorphically evaluating the two-party distributed decryption method from either the lsb case or the msb case given in Section 4.1 and 4.2, respectively, the final n messages m_i will sum up to the decryption of the ciphertext ct. The only difference is that instead of adding or subtracting a random nonce R, the parties are using the PRF F to randomize their shares in distributed decryption; thus, the correctness property of the scheme relies on the security of F.

Algorithm $\mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scooby}}(\mathsf{ct})$ **(for** lsb**-based construction)**

Let $F : \{0,1\}^\lambda \times [n] \to R_q$ be a pseudorandom function.

$\mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scooby}}(\mathsf{ct})$:

1. Parse sk_i as $\widetilde{\mathsf{sk}}_i$, K^{prf} and $(\mathsf{pk}_v^{\mathsf{FHE}}, \mathsf{ct}_v)$, for every node v from the root to leaf i.
2. Let $\widetilde{\mathsf{ct}}_\perp := \mathsf{ct}$.
3. For each internal node v on the path from the root to leaf i (excluding the root and leaf):
   (a) Write $v = u\|b$, where u is the parent of v (so $b = 0$ if v is a left child and $b = 1$ otherwise).
   (b) Define the function:
$$f_{\widetilde{\mathsf{ct}}_u}^b : \mathsf{sk} \mapsto \left(\widetilde{\mathsf{ct}}_u \cdot (b, -\mathsf{sk}) + (-1)^b \cdot F(K^{\mathsf{prf}}, u) \pmod q\right) \pmod p$$
   (c) Compute $\widetilde{\mathsf{ct}}_v := \mathsf{Eval}_{\mathsf{pk}_v}^{\mathsf{FHE}}(f_{\widetilde{\mathsf{ct}}_u}^b, \mathsf{ct}_v)$.
4. Write $i = u\|b$, then take the leaf ciphertext $\widetilde{\mathsf{ct}}_i$ and output the share
$$m_i = \left(\widetilde{\mathsf{ct}}_i \cdot (b, -\widetilde{\mathsf{sk}}_i) + (-1)^b \cdot F(K^{\mathsf{prf}}, u) \pmod q\right) \pmod p$$

Fig. 4. Decryption to shares for lsb-based Scooby.

Theorem 1. *Let F be a secure pseudorandom function, and suppose there is an LD-like FHE scheme which satisfies the hardness assumption from Definition 10, such that $(q, N, B, d, \Delta, S) \leftarrow \mathsf{ParamGen}(1^\lambda, p)$ with $q > 2 \cdot p \cdot (B+1) \cdot 2^{\mathsf{sec}}$. Then the* Scooby *construction in Fig. 3-4 is a correct and secure n-party homomorphic encryption scheme with decryption to shares.*

Proof. Let n be the number of parties. Let $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow \mathsf{SetUp}^{\mathsf{Scooby}}(\lambda, p, n)$. We need to show that Scooby, with the setup and decryption algorithms described in Figs. 3, 4, satisfies the definition of correctness and security given in Section 3. Correctness. Here, we only consider the lsb-based construction, the proof for the msb-based variant is similar.

We assume $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$ be the output of the setup algorithm. Each party P_i holds sk_i, corresponding to a leaf i in the binary tree generated in the setup algorithm. We say that a party P_i is *involved in a node* $\mathcal{N}$ of this binary tree if the path from the leaf i to the root contains the node $\mathcal{N}$.

Let ct be a ciphertext corresponding to public-key pk and encrypting a message m (possibly obtained through $\mathsf{Eval}_{\mathsf{pk}}^{\mathsf{Scooby}}$), we show that $m_i \leftarrow \mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scooby}}(\mathsf{ct}), i \in [n]$, satisfy $\sum_i m_i = m$, except with negligible probability.

During the decryption algorithm, and starting from the root, each party P_i, using its secret key sk_i, iteratively performs homomorphic evaluations of the 2-party distributed decryption function until it reaches the leaf i.

More concretely, in the first step at the root level, we have $\widetilde{\mathsf{ct}} := \mathsf{ct}$ and the 2-party decryption functions:

$$f_{\widetilde{\mathsf{ct}}}^0 : \mathsf{sk} \mapsto \left(\widetilde{\mathsf{ct}} \cdot (0, -\mathsf{sk}) + F(K^{\mathsf{prf}}) \pmod q\right) \pmod p$$
$$f_{\widetilde{\mathsf{ct}}}^1 : \mathsf{sk} \mapsto \left(\widetilde{\mathsf{ct}} \cdot (1, -\mathsf{sk}) - F(K^{\mathsf{prf}}) \pmod q\right) \pmod p.$$

Going a level down in the tree, each party P_i is involved either in the node $v = 0$ or in $v = 1$, hence it will compute either

$$\widetilde{\mathsf{ct}}_0 = \mathsf{Eval}_{\mathsf{pk}_0}^{\mathsf{FHE}}(f_{\widetilde{\mathsf{ct}}}^0, \mathsf{ct}_0) = \mathsf{Eval}_{\mathsf{pk}_0}^{\mathsf{FHE}}(f_{\widetilde{\mathsf{ct}}}^0, \mathsf{Enc}_{\mathsf{pk}_0}^{\mathsf{FHE}}(\widetilde{\mathsf{sk}_0}))$$

or

$$\widetilde{\mathsf{ct}}_1 = \mathsf{Eval}_{\mathsf{pk}_1}^{\mathsf{FHE}}(f_{\widetilde{\mathsf{ct}}}^1, \mathsf{ct}_1) = \mathsf{Eval}_{\mathsf{pk}_1}^{\mathsf{FHE}}(f_{\widetilde{\mathsf{ct}}}^1, \mathsf{Enc}_{\mathsf{pk}_1}^{\mathsf{FHE}}(\widetilde{\mathsf{sk}_0}))$$

Since $\widetilde{\mathsf{ct}}$ is an encryption under the public key pk corresponding to $\mathsf{sk} = \widetilde{\mathsf{sk}_0} + \widetilde{\mathsf{sk}_1}$, the two ciphertexts $\widetilde{\mathsf{ct}}_0, \widetilde{\mathsf{ct}}_1$ are the result of an homomorphic evaluation of the 2-party decryption functions $f_{\widetilde{\mathsf{ct}}}^0, f_{\widetilde{\mathsf{ct}}}^1$ and represent encryptions of m_0^1 and m_1^1 under pk_0 and pk_1, respectively, such that $m_0^1 + m_1^1 = m$.

Further following the path towards the leaf, each party will repeat the same procedure for at most one node per level, always obtaining an encryption of a share of m that they are not able to decrypt until they reach the level just above the leaves. In the last step, each P_i considers the leaf ciphertext and, performing a 2-party distributed decryption, outputs a share of m.

The correctness of the procedure directly follows from the fact that all the ciphertexts are randomized using the PRF and by Proposition 2. Each party involved in the decryption holds all the necessary encryptions and secret-key material to be able to successfully conclude the algorithm with probability at least $1 - N \cdot 2^{-\mathsf{sec}}$. The latter probability arising as we

have N coefficients which need to be decrypted correctly, which happens with probability of $1 - 2^{-\mathsf{sec}}$, via the method in Section 4.1, or a similar method, which we present in the full version.

Therefore, the overall probability that all the parties are correct is $1 - n \cdot N \cdot 2^{-\mathsf{sec}}$.

Security. We consider the strongest case of $n-1$ corruptions, so the adversary is given all-but-one of the secret keys sk_i. We need to show that the resulting encryption scheme is IND-CPA secure. Let j be the index of the honest party. We consider an experiment where the adversary is given a public key pk together with the secret keys $\{\mathsf{sk}_i\}_{i \neq j}$ and the encryption of a challenge message m. We show that this is indistinguishable from an experiment where the adversary receives an encryption of a random message. We proceed via a hybrid argument, starting with the bottom layer h of the tree.

Hybrid H_h This is the experiment defined above, where the adversary is given pk, $\{\mathsf{sk}_i\}_{i \neq j}$ and an encryption of m.

Hybrid H_ℓ, for $\ell \in \{h-1, \ldots, 1\}$ Let v be the index of the node on layer ℓ of the tree that lies on the path to leaf j. This hybrid is defined as in $H_{\ell+1}$, except here we modify the ciphertext ct_v, which was previously an encryption of the secret key share $\widetilde{\mathsf{sk}}_v$ under pk_v, and replace it with an encryption of zero under pk_v.

Hybrid H_0 This hybrid is defined to be the same as H_1, except we replace the ciphertext ct, which previously encrypted the message m under the root key $\mathsf{pk}_\perp$, with an encryption of zero. This hybrid is now independent of m.

We now argue indistinguishability. For each $\ell \in [h]$, hybrids H_ℓ and $H_{\ell-1}$ are computationally indistinguishable due to the bounded IND-CPA security property of the LD-based FHE scheme (Definition 10). We can apply this property because in hybrid $\ell - 1$ at node u, it always holds that one of the secret keys $\widetilde{\mathsf{sk}}_{u\|0}$ or $\widetilde{\mathsf{sk}}_{u\|1}$ is unknown to the adversary, as required. Furthermore, there is no circular security issue, because the encrypted secret key $\widetilde{\mathsf{sk}}_u$ is sampled independently of the keys $\widetilde{\mathsf{sk}}_{u\|0}$ and $\widetilde{\mathsf{sk}}_{u\|1}$. □

Remark 2. Note that for correctness to hold it is not sufficient that for a single party the path from the root to the node is correctly split. We need this to happen for *all* parties simultaneously. This means that the obtained probability is in fact $1 - n \cdot N \cdot 2^{-\mathsf{sec}}$ and not, as initially might be believed, $1 - \log(n) \cdot N \cdot 2^{-\mathsf{sec}}$.

5.3.1. A simpler variant relying on circular security

The previous construction avoids relying on a circular security assumption by switching to a freshly sampled FHE key at each node of the tree. We could instead simplify this slightly, with a variant of the construction where only one set of FHE secret keys is used. Here, we would start by sampling an independent secret key $\widetilde{\mathsf{sk}}_i$ for each leaf i. The public key associated with node v is then defined as $\mathsf{pk}_v = \mathsf{PubKeyGen}(1^\lambda, \mathsf{sk}_v, \mathsf{params})$, where sk_v is the sum of all the leaf secret keys that are descendants of v. We additionally encrypt sk under pk_v and give this out to the relevant parties. This introduces a circular security assumption, however, it does not seem to offer any significant efficiency benefits except for a slightly simpler setup algorithm.

5.4. BGV parameters supporting Scooby

It would appear that at first sight the parameters needed for Scooby are larger than those needed for standard FHE bootstrapping, due to the increase in q required by Equation (1). However, this is not necessarily the case, as we now explain in the case of the BGV encryption scheme.

Standard BGV decryption simply requires the bound $q > 2 \cdot p \cdot (B+1)$ for valid decryption, so we appear to have boosted the size of q by a factor of 2^{sec}. However, bootstrappable BGV as implemented in (say) HELib [22] utilizes an underlying levelled SHE scheme. At level zero, where no further homomorphic operations may take place without bootstrapping, we have a ciphertext modulus q_0 which satisfies $q_0 > 2 \cdot p \cdot (B+1)$. At level L, i.e., the initial encryption level, we have a ciphertext modulus q_L which satisfies $q_L > 2 \cdot p \cdot (B+1) \cdot 2^{b_p \cdot L}$, where b_p is the (average) bits-per-level of the chain of ciphertext moduli. On passing from each level from L down to zero, the size of the ciphertext modulus drops by (on average) 2^{b_p}. Note that, when bootstrapping a ciphertext from level zero, we do not end up with a ciphertext at level L, instead we obtain a ciphertext at level U (which denotes the so-called "usable" number of levels).

To see how this affects Scooby, we need to remember that at the end of the $\mathsf{Eval}_{\mathsf{pk}}^{\mathsf{Scooby}}$ procedure we will have a ciphertext at level U. This will satisfy our bound in Equation (1) if $2^{b_p \cdot U} \geq 2^{\mathsf{sec}}$. Then, in executing $\mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scooby}}$, at each level of the tree we notice that we are actually executing an operation equivalent to bootstrapping. This is because at each node $v = u\|b$, where u is the parent node and $b \in \{0,1\}$, we are essentially either performing a homomorphic decryption with the key $(1, -\widetilde{\mathsf{sk}}_{u\|1}^{\mathsf{FHE}})$, or a homomorphic decryption with the key $(0, -\widetilde{\mathsf{sk}}_{u\|0}^{\mathsf{FHE}})$. Thus, at each stage of the execution of $\mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scooby}}$ we have a ciphertext ct which is at level U.

Examining the bootstrappable BGV parameters proposed in [22] we see that in all cases we have $2^{b_p \cdot U} \geq 2^{128}$. Thus the Equation (1) does not actually result in any increase in parameters, at least in the case of the BGV scheme.

Scrappy: n-party HEDS from FHE + HSS

Let $(\mathsf{KeyGen}^{\mathsf{FHE}}, \mathsf{Enc}^{\mathsf{FHE}}, \mathsf{Eval}^{\mathsf{FHE}}, \mathsf{Dec}^{\mathsf{FHE}})$ be an FHE scheme and $(\mathsf{Share}^{\mathsf{HSS}}, \mathsf{Eval}^{\mathsf{HSS}})$ be a 2-party HSS scheme for the FHE decryption circuit.

- $\mathsf{SetUp}^{\mathsf{Scrappy}}(1^\lambda, n)$: Construct a complete binary tree of height $h = \lceil \log n \rceil$ and n leaves, and index the levels from 0 (at the root) up to h. Each node in level i of the tree is labelled with a string of i bits, so the root is labelled with the empty string $\perp$, and the children of node v are labelled $v\|0$ and $v\|1$.
  1. Sample a root key pair $(\mathsf{pk}_\perp, \mathsf{sk}_\perp) = \mathsf{KeyGen}^{\mathsf{FHE}}(1^\lambda)$.
  2. Sample HSS shares $(s_0, s_1) = \mathsf{Share}^{\mathsf{HSS}}(\mathsf{sk}_\perp)$.
  3. For each internal node v (excluding the root and leaf nodes), with parent node u and children $v\|0, v\|1$, compute the following values:
     (a) $(\mathsf{pk}_v, \mathsf{sk}_v) = \mathsf{KeyGen}^{\mathsf{FHE}}(1^\lambda)$.
     (b) $\mathsf{ct}_v^s = \mathsf{Enc}_{\mathsf{pk}_v}^{\mathsf{FHE}}(s_v)$
     (c) $(s_{v\|0}, s_{v\|1}) = \mathsf{Share}^{\mathsf{HSS}}(\mathsf{sk}_v)$.
  4. Output $\mathsf{pk} := \mathsf{pk}_\perp$ and the secret keys $\mathsf{sk}_i := \left(s_i, \{\mathsf{pk}_v, \mathsf{ct}_v^s\}_{v \in \mathsf{pathTo}(i)}\right)$, for $i = 0, \ldots, n-1$.
- $\mathsf{Enc}_{\mathsf{pk}}^{\mathsf{Scrappy}}(m)$: Output $\mathsf{ct} = \mathsf{Enc}_{\mathsf{pk}}^{\mathsf{FHE}}(m)$.
- $\mathsf{Eval}_{\mathsf{pk}}^{\mathsf{Scrappy}}(C, (\mathsf{ct}_1, \ldots, \mathsf{ct}_m))$: Output $\widetilde{\mathsf{ct}} = \mathsf{Eval}_{\mathsf{pk}}^{\mathsf{FHE}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_m)$.
- $\mathsf{Dec}_{\mathsf{sk}_i}^{\mathsf{Scrappy}}(\widetilde{\mathsf{ct}})$:
  1. Let $\widetilde{\mathsf{ct}}_\perp := \widetilde{\mathsf{ct}}$.
  2. For each internal node v on the path from the root to leaf i (excluding the root and leaf), with parent node u:
     (a) Define the pair of functions:
     $$f_{\widetilde{\mathsf{ct}}_u} : \mathsf{sk} \mapsto \mathsf{Dec}_{\mathsf{sk}}^{\mathsf{FHE}}(\widetilde{\mathsf{ct}}_u)$$
     $$g_{\widetilde{\mathsf{ct}}_u} : s_v \mapsto \mathsf{Eval}^{\mathsf{HSS}}(f_{\widetilde{\mathsf{ct}}_u}, s_v)$$
     (b) Compute $\widetilde{\mathsf{ct}}_v := \mathsf{Eval}_{\mathsf{pk}_v}^{\mathsf{FHE}}(g_{\widetilde{\mathsf{ct}}_u}, \mathsf{ct}_v^s)$.
  3. Output $y_i = \mathsf{Eval}^{\mathsf{HSS}}(f_{\mathsf{ct}}, s_i)$.

Fig. 5. Constructing n-party HEDS using standard FHE and 2-party HSS.

6. Multi-party HEDS from weaker assumptions

We now present alternative constructions to the previous section, without relying on FHE with linear decryption and a super-polynomial modulus. In the first construction, in Section 6.1, we use any generic FHE scheme and a 2-party HSS scheme that supports homomorphic evaluation of the FHE decryption circuit. This means we no longer need the local decryption trick from Section 4.1, so can use FHE based on LWE with a polynomial modulus [12]. All LWE-based FHE constructions have decryption in NC1, so the 2-party HSS can be instantiated based on the Paillier assumption [25] or on class groups [1], which support HSS for all of NC1.

In Section 6.2, we also give a variant of the construction that *only* requires 2-party HSS, and not FHE. This gives a way to bootstrap two-party HSS constructions to the multi-party setting. We show how it can be used to transform two-party HSS for branching programs, based on Paillier encryption, into 4-party HSS for homomorphic evaluation of constant-degree polynomials.

6.1. Scrappy: HEDS from standard FHE + HSS for NC1

This construction, shown in Fig. 5, follows the tree-based structure of Scooby from the previous section. Previously, though, at each node of the tree, an FHE ciphertext was split into two ciphertexts encrypting shares of the message, by doing a special homomorphic decryption procedure tailored to the linear decryption property of the FHE scheme. In Scrappy, we instead do the homomorphic decryption procedure inside a 2-party HSS scheme. Since most FHE schemes have decryption in NC1, it suffices to rely on HSS for NC1, which can be built from non-LWE-based assumptions. Of course, if done naively, this means we no longer get encrypted shares of the previous message, but would actually obtain the shares directly due to use of HSS. To avoid leaking all intermediate shares, we use an additional FHE scheme on each level of the tree, and use this to homomorphically evaluate the HSS evaluation procedure. The HSS evaluation keys are then only given out at the leaves of the tree, while at higher levels they are encrypted under FHE. Note that we only need the weaker, private-key form HSS, from Remark 1, where the sharing algorithm can be seen as done by a trusted dealer.

Theorem 2. *Suppose there exists fully homomorphic encryption, and a 2-party, secure HSS scheme that supports homomorphic evaluation of the FHE scheme's decryption circuit. Then, the construction in Fig. 5 is a correct and secure n-party homomorphic encryption scheme with decryption to shares, for any* $n = \mathsf{poly}(\lambda)$.

Proof. CORRECTNESS. We need to show that for any messages $m_1, \ldots, m_\rho$ and ciphertext $\widetilde{\mathsf{ct}}$ that is the output of Eval, on input a circuit C and encryptions of $m_1, \ldots, m_\rho$, we have

$$y_1 + \cdots + y_n = C(m_1, \ldots, m_\rho) \mod p$$

where $y_i = \mathsf{Dec}^{\mathsf{Scrappy}}_{\mathsf{sk}_i}(\widetilde{\mathsf{ct}})$ and sk_i is the i-th secret key.

Consider the first step of the decryption algorithm, corresponding to the left and right children of the root node. For some $v \in \{0,1\}$, this step homomorphically evaluates (in FHE) the function

$$g_{\widetilde{\mathsf{ct}}} : s_v \mapsto \mathsf{Eval}^{\mathsf{HSS}}(f_{\widetilde{\mathsf{ct}}}, s_v)$$

where here, $f_{\widetilde{\mathsf{ct}}}$ takes as input an FHE secret key, and uses it to decrypt $\widetilde{\mathsf{ct}}$. Note that this step is performed on the FHE ciphertext ct^s_v, where $v = 0$ for party i with $i < 2^h/2$, and $v = 1$ for the remaining parties. The parties obtain respective ciphertexts $\widetilde{\mathsf{ct}}_0, \widetilde{\mathsf{ct}}_1$, encrypted under pk_0 or pk_1.

Let $\widetilde{m}_0, \widetilde{m}_1$ denote the messages in these two ciphertexts. Since s_0, s_1 are HSS shares of the FHE key $\mathsf{sk}_\perp$, by the correctness of FHE and HSS, except with negligible probability it holds that

$$\begin{aligned}
\widetilde{m}_0 + \widetilde{m}_1 &= g_{\widetilde{\mathsf{ct}}}(s_0) + g_{\widetilde{\mathsf{ct}}}(s_1) \\
&= \mathsf{Eval}^{\mathsf{HSS}}(f_{\widetilde{\mathsf{ct}}}, s_0) + \mathsf{Eval}^{\mathsf{HSS}}(f_{\widetilde{\mathsf{ct}}}, s_1) = f_{\widetilde{\mathsf{ct}}}(\mathsf{sk}_\perp) \\
&= \mathsf{Dec}^{\mathsf{FHE}}_{\mathsf{sk}_\perp}(\widetilde{\mathsf{ct}}) = \mathsf{Dec}^{\mathsf{FHE}}_{\mathsf{sk}_\perp}(\mathsf{Eval}^{\mathsf{FHE}}_{\mathsf{pk}_\perp}(\mathsf{Enc}^{\mathsf{FHE}}_{\mathsf{pk}_\perp}(m_1), \ldots, \mathsf{Enc}^{\mathsf{FHE}}_{\mathsf{pk}_\perp}(m_\rho))) \\
&= \underbrace{C(m_1, \ldots, m_\rho)}_{=\widetilde{m}}
\end{aligned}$$

This shows that, after the first layer of evaluation, the partition of parties $(P_0, \ldots, P_{2^h/2-1})$ and $(P_{2^h/2}, \ldots, P_{n-1})$ will each have obtained an encryption of $\widetilde{m}_0$ or $\widetilde{m}_1$, under pk_0 or pk_1, which are shares of the correct message $\widetilde{m}$. By the same argument as above, at each subsequent node, the encryption of $\widetilde{m}_b$ will be split into two further encrypted shares, encrypted under the public keys for the next layer. This process continues until the penultimate layer of the tree, where for each non-leaf node v, any party $i \in \{v\|0, v\|1\}$ will compute a ciphertext $\widetilde{\mathsf{ct}}_v$, which encrypts under pk_v a share $\widetilde{m}_v$.

At the final layer of the tree, if v has two children $v\|0, v\|1$, then each of the parties labelled with these leaves holds an HSS share s_i of sk_v, which can be used to homomorphically decrypt $\widetilde{\mathsf{ct}}_v$, where v is the parent of i, splitting $\widetilde{m}_v$ into two last shares. Otherwise, if v itself is a leaf node, then the party labelled with v uses its HSS share s_v to recover its share directly.

This shows that, with overwhelming probability, the parties obtain a correct sharing of the result.

SECURITY: We argue security for the strongest case of $n-1$ corruptions, and assume for simplicity that n is a power of two. Let j be the index of the honest party. We consider an experiment where the adversary is given a public key pk together with the secret keys $\{\mathsf{sk}_i\}_{i \neq j}$, and the encryption of a challenge message m. We gradually define a sequence of hybrid experiments, starting from the bottom layer h of the tree, where we eventually end up with one that is independent of the message m and indistinguishable from the first experiment. This implies the security property from Definition 8.

Hybrid $H_{h,1}$: This is the experiment defined above, where the adversary is given pk, $\{\mathsf{sk}_i\}_{i \neq j}$ and an encryption of m.

Hybrids $H_{\ell,0}$ *and* $H_{\ell-1,1}$*, for* $\ell = h, \ldots, 1$: Let v be the index of the node on layer ℓ of the tree that lies on $\mathsf{pathTo}(j)$, let $\overline{v}$ be its sibling and u their parent node. These hybrids are as follows:

- $H_{\ell,0}$: This hybrid is defined as in $H_{\ell,1}$, except we replace the HSS share $s_{\overline{v}}$, which was originally computed with $\mathsf{Share}^{\mathsf{HSS}}(\mathsf{sk}_u)$, with a share from $\mathsf{Share}^{\mathsf{HSS}}(0)$. If v is not a leaf node, the ciphertext $\mathsf{ct}_{\overline{v}}$ is now computed as an encryption of the new share $s_{\overline{v}}$.
- $H_{\ell-1,1}$ This hybrid is defined as in $H_{\ell,0}$, except here we modify the ciphertext ct_u, which was either an encryption of the HSS share s_u under pk_u, or (if $u = \perp$ is the root) an encryption of m under $\mathsf{pk}_\perp$. In either case, we replace it with an encryption of zero under pk_u.

Hybrid $H_{\ell,0}$ is indistinguishable from $H_{\ell,1}$, because the second HSS share s_v is independent of the view of the adversary, so we can rely on the security property of HSS. Hybrid $H_{\ell-1,1}$ is indistinguishable from $H_{\ell,0}$, due to the IND-CPA security of the FHE scheme; this relies on the fact that in $H_{\ell,0}$, we have removed the FHE secret key sk_u from the view of the adversary.

Construction n^2-HEDS

Let n-HEDS be an n-party HEDS. We build n^2-party HEDS, and label the parties $P_{i,j}$, for $i, j \in [n]$

- $\mathsf{SetUp}^{n^2\text{-HEDS}}(1^\lambda, n^2)$:
  1. Let $(\mathsf{pk}_\perp, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) = \mathsf{SetUp}^{n\text{-HEDS}}(1^\lambda, n)$.
  2. For $i \in [n]$:
     (a) Sample $(\mathsf{pk}_i, \mathsf{sk}_{i,1}, \ldots, \mathsf{sk}_{i,n}) = \mathsf{SetUp}^{n\text{-HEDS}}(1^\lambda, n)$.
     (b) Sample $\mathsf{ct}_i^s = \mathsf{Enc}_{\mathsf{pk}_i}^{n\text{-HEDS}}(\mathsf{sk}_i)$.
  3. Output $\mathsf{pk} := (\mathsf{pk}_\perp, \mathsf{ct}_1^s, \ldots, \mathsf{ct}_n^s)$ and the n^2 secret keys $\mathsf{sk}_{i,j} := (\mathsf{ct}_i^s, \mathsf{sk}_{i,j}, \mathsf{pk}_i)$, for $i, j \in [n]$.
- $\mathsf{Enc}_{\mathsf{pk}}^{n^2\text{-HEDS}}(m)$: Output $\mathsf{ct} = \mathsf{Enc}_{\mathsf{pk}_\perp}^{n\text{-HEDS}}(m)$.
- $\mathsf{Eval}_{\mathsf{pk}}^{n^2\text{-HEDS}}(C, (\mathsf{ct}_1, \ldots, \mathsf{ct}_\rho))$:
  1. Compute $\mathsf{ct} = \mathsf{Eval}_{\mathsf{pk}_\perp}^{n\text{-HEDS}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_\rho)$.
  2. Let f_{ct} be the function that takes as input sk_i and outputs $\mathsf{Dec}_{\mathsf{sk}_i}^{n\text{-HEDS}}(\mathsf{ct})$.
  3. Compute $\widetilde{\mathsf{ct}}_i = \mathsf{Eval}_{\mathsf{pk}_i}^{n\text{-HEDS}}(f_{\mathsf{ct}}, \mathsf{ct}_i^s)$, for $i = 1, \ldots, n$.
  4. Output $(\widetilde{\mathsf{ct}}_1, \ldots, \widetilde{\mathsf{ct}}_n)$.
- $\mathsf{Dec}_{\mathsf{sk}_{i,j}}^{n^2\text{-HEDS}}(\widetilde{\mathsf{ct}}_i)$: Output $y_{i,j} = \mathsf{Dec}_{\mathsf{sk}_{i,j}}^{n\text{-HEDS}}(\widetilde{\mathsf{ct}}_i)$.

Fig. 6. Bootstrapping n-party HEDS to n^2 parties.

In the final hybrid $H_{0,1}$, the root ciphertext $\mathsf{ct}_\perp$ has been replaced with an encryption of zero, so the view of the adversary is independent of the original message m. This concludes the proof. □

The above theorem implies n-party HEDS assuming (1) LWE with a polynomial modulus [12], (2) circular security, and (3) HSS for NC1 circuits, which can be based on decisional composite residuosity [25] or a DDH-like assumption in class groups [1]. If we only require n-party HEDS for bounded-depth circuits, we can remove the circular security assumption, since we only required levelled FHE.

6.2. Shaggy: *bootstrapping HEDS to more parties*

We now give a separate transformation that increases the number of parties in HEDS, *without* relying on fully homomorphic encryption. This construction, in Fig. 6, essentially applies one layer of the previous, tree-based construction, with a branching factor of n instead of 2. Additionally, instead of alternating between FHE and HSS evaluation, we always evaluate within an n-party HEDS scheme. This allows bootstrapping any sufficiently powerful n-party HEDS to support n^2 parties.

Theorem 3. *Let n-HEDS be a correct and secure, n-party HEDS for a class of circuits $\mathcal{C}$, whose decryption algorithm, when viewed as a function of sk_i, can be written as a circuit in $\mathcal{C}$. Then, n^2-HEDS (in Fig. 6) is a correct and secure n^2-party HEDS for $\mathcal{C}$.*

Observe that in n^2-HEDS the encryption and evaluation algorithms are the same as in n-Scooby, while the complexity of decryption increases by a polynomial factor.

Proof. Correctness: Let $\mathsf{ct}_1, \ldots, \mathsf{ct}_\rho$ be n^2-HEDS encryptions of messages $m_1, \ldots, m_\rho$. During decryption, party (i, j) first computes a ciphertext $\widetilde{\mathsf{ct}}_i$, a homomorphic decryption of ct, now encrypted under pk_i, and then uses $\mathsf{sk}_{i,j}$ to recover $y_{i,j}$, a share of this.

By the correctness of n-HEDS, for each $i \in [n]$ we have

$$\begin{aligned}\sum_{j=1}^{n} y_{i,j} &= \sum_{j} \mathsf{Dec}_{\mathsf{sk}_{i,j}}^{n\text{-HEDS}}(\mathsf{Eval}_{\mathsf{pk}_i}^{n\text{-HEDS}}(f_{\mathsf{ct}}, \mathsf{ct}_i^s)) \\ &= f_{\mathsf{ct}}(\mathsf{sk}_i) \\ &= \mathsf{Dec}_{\mathsf{sk}_i}^{n\text{-HEDS}}(\mathsf{ct})\end{aligned}$$

It follows, again due to the correctness of n-HEDS, that $\sum_{i=1}^{n}\sum_{j=1}^{n} y_{i,j} = \sum_i \mathsf{Dec}_{\mathsf{sk}_i}^{n\text{-HEDS}}(\mathsf{ct}) = C(m_1, \ldots, m_\rho)$, as required.

Security: The security can be argued in a similar way to the proof of Theorem 2, for just a single layer of the tree. Since the adversary corrupts at most $n^2 - 1$ parties, there is at least one pair (i, j) where the adversary does not know $\mathsf{sk}_{i,j}$. This means that we can replace the ciphertext ct_i^s by an encryption of zero, which will be secure due to the security of

n-HEDS. This in turn means that sk_i is unknown to the adversary, so we can consider another hybrid where the challenge ciphertext ct is replaced with an encryption of zero. This implies IND-CPA security to an adversary who sees up to $n^2 - 1$ secret keys. □

Note that the decryption complexity of the bootstrapped construction n^2-HEDS is increased by a polynomial factor. Depending on the original n-party scheme, then, it may not be possible to apply the transformation more than once, if the new decryption algorithm is no longer in the class $\mathcal{C}$.

6.2.1. Shaggy*: instantiation with HSS from Paillier*

We now describe the Shaggy construction, which is obtained by applying the above transformation to two-party HSS based on the decisional composite residuosity assumption used in Paillier encryption. We can only apply the transformation once, so we obtain a 4-party HEDS/HSS scheme, which can support homomorphic evaluation of constant-degree polynomials. As the underlying two-party scheme n-HEDS, we use the HSS construction from [25] or [27].

First, we need to frame the 2-party HSS constructions of [25,27] in our HEDS framework. The constructions are given in a "public-key" flavour of HSS, with $\mathsf{SetUp}^{\mathsf{HSS}}$ and $\mathsf{Enc}^{\mathsf{HSS}}$ algorithms which are the same as in HEDS. The $\mathsf{Eval}^{\mathsf{HSS}}$ algorithm, however, requires knowing a secret key, unlike the syntax for $\mathsf{Eval}^{\mathsf{HEDS}}$. We can still make this fit in our HEDS framework, by adjusting the scheme so that homomorphic evaluation is performed in the $\mathsf{Dec}^{\mathsf{HEDS}}_{\mathsf{sk}_i}$ step, which knows the secret key, instead of $\mathsf{Eval}^{\mathsf{HEDS}}$. Concretely, we define the $\mathsf{Eval}^{\mathsf{HEDS}}$ algorithm to simply output an "evaluated ciphertext" defined to be the set of input ciphertexts together with the circuit C. These are then passed to $\mathsf{Dec}^{\mathsf{HEDS}}$, together with the secret key, which then runs the HSS evaluation algorithm. This makes the resulting HEDS non-compact, since the complexity of decryption depends on the circuit, but it can still be used for the construction in Fig. 6.

Complexity of evaluation in Paillier-based HEDS We now analyze the circuit complexity of the resulting $\mathsf{Dec}^{\mathsf{HEDS}}$ algorithm, which performs HSS evaluation of constant-degree polynomials. We can assume the polynomial is a simple monomial $f(x_1, \ldots, x_c) = x_1 x_2 \cdots x_c$ for a constant number of inputs (since to handle sums of monomials, it's enough to evaluate each monomial separately and add the shares).

With the methods of [25,27], each input x_i is given as a Paillier encryption of x_i, together with encryptions of x_i multiplied with each bit of the secret key. In homomorphic evaluation, the parties perform $c - 1$ sequential multiplications, where in each of these, the core operation is a step that computes:

$$z = \mathsf{DDLog}(C^d \mod N^2) + F_k(\mathsf{id}) \mod N$$

Here, $N = pq$ is a public modulus, $C \in \mathbb{Z}^*_{N^2}$ is a ciphertext, d is a secret share that is known only to one party, and F is a pseudorandom function with key k known to both parties. The distributed discrete log function $\mathsf{DDLog}(X)$ computes $\lfloor X/N \rfloor \cdot (X \bmod N)^{-1} \bmod N$.

In general, modular exponentiation and inversion are not known to be in NC1. However, it turns out that $\mathsf{DDLog}(C^d)$, when viewed as a function of d for fixed C, does lie in NC1. The idea is that since C is public, we can consider the powers $C^{2^j} \bmod N^2$ as hard-coded into the description of the function. Similarly, we hardcode $C^{-2^j} \bmod N$, for $j = 1, \ldots, \ell$, where ℓ is the bit length of d. This allows computing

$$C^d = \prod_{j=1}^{\ell} (C^{2^j})^{d_j} \bmod N^2, \quad (C^d \bmod N)^{-1} = \prod_{j=1}^{\ell} (C^{-2^j})^{d_j} \bmod N$$

Since iterated product, modular reduction, addition/subtraction and integer division are all in NC1 [2], $\mathsf{DDLog}(C^d)$ can be computed as an NC1 circuit. Furthermore, evaluation of a PRF based on factoring can be done in NC1 [24].

In the complete multiplication algorithm, the above step is repeated $O(\lambda)$ times in parallel, which does not affect the circuit depth. The multiplication algorithm is run c times sequentially, where the outputs of one multiplication are used as the private d shares input to the next. If c is a constant, it follows that the entire evaluation procedure is in NC1.

Plugging in two-party HSS for poly-sized branching programs (which includes NC1), we obtain the following.

Corollary 1. *Assume the decisional composite residuosity assumption holds. Then, there exists a 4-party (non-compact) homomorphic encryption scheme with decryption to shares for constant-degree polynomials.*

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: All authors reports writing assistance was provided by Nigel P. Smart (imec-COSIC, KU Leuven, Belgium).

References

[1] D. Abram, I. Damgård, C. Orlandi, P. Scholl, An algebraic framework for silent preprocessing with trustless setup and active security, Cryptology ePrint Archive, Report 2022/363, https://ia.cr/2022/363, 2022.

[2] P. Beame, S. Cook, H. Hoover, Log depth circuits for division and related problems, in: 25th Annual Symposium on Foundations of Computer Science, 1984, 1984, pp. 1–6.

[3] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, P. Scholl, Efficient pseudorandom correlation generators: silent OT extension and more, in: A. Boldyreva, D. Micciancio (Eds.), CRYPTO 2019, Part III, in: LNCS, vol. 11694, Springer, Heidelberg, Aug 2019, pp. 489–518.

[4] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, P. Scholl, Correlated pseudorandom functions from variable-density LPN, in: 61st FOCS, IEEE Computer Society Press, Nov 2020, pp. 1069–1080.

[5] E. Boyle, N. Gilboa, Y. Ishai, Function secret sharing, in: E. Oswald, M. Fischlin (Eds.), EUROCRYPT 2015, Part II, in: LNCS, vol. 9057, Springer, Heidelberg, Apr 2015, pp. 337–367.

[6] E. Boyle, N. Gilboa, Y. Ishai, Breaking the circuit size barrier for secure computation under DDH, in: M. Robshaw, J. Katz (Eds.), CRYPTO 2016, Part I, in: LNCS, vol. 9814, Springer, Heidelberg, Aug 2016, pp. 509–539.

[7] E. Boyle, N. Gilboa, Y. Ishai, Function secret sharing: improvements and extensions, in: E.R. Weippl, S. Katzenbeisser, C. Kruegel, A.C. Myers, S. Halevi (Eds.), ACM CCS 2016, ACM Press, Oct 2016, pp. 1292–1303.

[8] E. Boyle, N. Gilboa, Y. Ishai, H. Lin, S. Tessaro, Foundations of homomorphic secret sharing, in: A.R. Karlin (Ed.), ITCS 2018, in: LIPIcs, vol. 94, Jan 2018, 21.

[9] E. Boyle, L. Kohl, P. Scholl, Homomorphic secret sharing from lattices without FHE, in: Y. Ishai, V. Rijmen (Eds.), EUROCRYPT 2019, Part II, in: LNCS, vol. 11477, Springer, Heidelberg, May 2019, pp. 3–33.

[10] Z. Brakerski, N. Döttling, S. Garg, G. Malavolta, Leveraging linear decryption: rate-1 fully-homomorphic encryption and time-lock puzzles, in: D. Hofheinz, A. Rosen (Eds.), TCC 2019, Part II, in: LNCS, vol. 11892, Springer, Heidelberg, Dec 2019, pp. 407–437.

[11] Z. Brakerski, C. Gentry, V. Vaikuntanathan, (Leveled) fully homomorphic encryption without bootstrapping, in: S. Goldwasser (Ed.), ITCS 2012, ACM, Jan 2012, pp. 309–325.

[12] Z. Brakerski, V. Vaikuntanathan, Lattice-based FHE as secure as PKE, in: M. Naor (Ed.), ITCS 2014, ACM, Jan 2014, pp. 1–12.

[13] I. Chillotti, N. Gama, M. Georgieva, M. Izabachène, Faster fully homomorphic encryption: bootstrapping in less than 0.1 seconds, in: J.H. Cheon, T. Takagi (Eds.), ASIACRYPT 2016, Part I, in: LNCS, vol. 10031, Springer, Heidelberg, Dec 2016, pp. 3–33.

[14] I. Chillotti, N. Gama, M. Georgieva, M. Izabachène, TFHE: fast fully homomorphic encryption over the torus, J. Cryptol. 33 (1) (Jan 2020) 34–91, https://doi.org/10.1007/s00145-019-09319-x.

[15] M. Clear, C. McGoldrick, Multi-identity and multi-key leveled FHE from learning with errors, in: R. Gennaro, M.J.B. Robshaw (Eds.), CRYPTO 2015, Part II, in: LNCS, vol. 9216, Springer, Heidelberg, Aug 2015, pp. 630–656.

[16] Y. Dodis, S. Halevi, R.D. Rothblum, D. Wichs, Spooky encryption and its applications, in: M. Robshaw, J. Katz (Eds.), CRYPTO 2016, Part III, in: LNCS, vol. 9816, Springer, Heidelberg, Aug 2016, pp. 93–122.

[17] J. Doerner, A. Shelat, Scaling ORAM for secure computation, in: B.M. Thuraisingham, D. Evans, T. Malkin, D. Xu (Eds.), ACM CCS 2017, ACM Press, Oct/Nov 2017, pp. 523–535.

[18] J. Fan, F. Vercauteren, Somewhat practical fully homomorphic encryption, Cryptology ePrint Archive, Report 2012/144, https://eprint.iacr.org/2012/144, 2012.

[19] C. Gentry, S. Halevi, Compressible FHE with applications to PIR, in: D. Hofheinz, A. Rosen (Eds.), TCC 2019, Part II, in: LNCS, vol. 11892, Springer, Heidelberg, Dec 2019, pp. 438–464.

[20] C. Gentry, A. Sahai, B. Waters, Homomorphic encryption from learning with errors: conceptually-simpler, asymptotically-faster, attribute-based, in: R. Canetti, J.A. Garay (Eds.), CRYPTO 2013, Part I, in: LNCS, vol. 8042, Springer, Heidelberg, Aug 2013, pp. 75–92.

[21] N. Gilboa, Y. Ishai, Distributed point functions and their applications, in: P.Q. Nguyen, E. Oswald (Eds.), EUROCRYPT 2014, in: LNCS, vol. 8441, Springer, Heidelberg, May 2014, pp. 640–658.

[22] S. Halevi, V. Shoup, Bootstrapping for HElib, J. Cryptol. 34 (1) (Jan 2021) 7, https://doi.org/10.1007/s00145-020-09368-7.

[23] P. Mukherjee, D. Wichs, Two round multiparty computation via multi-key FHE, in: M. Fischlin, J.S. Coron (Eds.), EUROCRYPT 2016, Part II, in: LNCS, vol. 9666, Springer, Heidelberg, May 2016, pp. 735–763.

[24] M. Naor, O. Reingold, Number-theoretic constructions of efficient pseudo-random functions, J. ACM 51 (2) (2004) 231–262.

[25] C. Orlandi, P. Scholl, S. Yakoubov, The rise of Paillier: homomorphic secret sharing and public-key silent OT, in: A. Canteaut, F.X. Standaert (Eds.), EUROCRYPT 2021, Part I, in: LNCS, vol. 12696, Springer, Heidelberg, Oct 2021, pp. 678–708.

[26] D. Rotaru, N.P. Smart, T. Tanguy, F. Vercauteren, T. Wood, Actively secure setup for SPDZ, J. Cryptol. 35 (1) (Jan 2022) 5, https://doi.org/10.1007/s00145-021-09416-w.

[27] L. Roy, J. Singh, Large message homomorphic secret sharing from DCR and applications, in: T. Malkin, C. Peikert (Eds.), CRYPTO 2021, Part III, in: LNCS, vol. 12827, Springer, Heidelberg, Aug 2021, pp. 687–717, Virtual Event.